



পাস বুক - ডাটা স্ট্রাকচার অ্যান্ড অ্যালগরিদম



পড়ার সময় : মাত্র ৮ ঘণ্টা



লক্ষ্য : ৩৫-৪০ মার্কস



ডিপ্লোমা ইঞ্জিনিয়ারিং শিক্ষার্থীদের জন্য দ্রুত রিভিশনের পারফেক্ট সলিউশন!



পৃষ্ঠা সংখ্যা :



ই-বুক ভার্সন : ১১০ পৃষ্ঠা



প্রিন্টযোগ্য কাগজে : ৩৭ পৃষ্ঠা



পরীক্ষা সংক্রান্ত তথ্য :



মোট মার্কস : ৬০



পাস মার্কস : ২৪



প্রশ্ন ও মার্কস বিশ্লেষণ :



অতিসংক্ষিপ্ত প্রশ্ন :

মোট পড়তে হবে : ৫০ টি

নিশ্চিত কমন : মিনিমাম ৫-৭ টি

নিশ্চিত মার্কস : ৫-৭ (প্রতি প্রশ্নে ১ মার্কস)



**সংক্ষিপ্ত প্রশ্ন :**

মোট পড়তে হবে : ৪০ টি

নিশ্চিত কমন : ৫-৭ টি

নিশ্চিত মার্কস : ১০-১৪ (প্রতি প্রশ্নে ২ মার্কস)

**রচনামূলক প্রশ্ন :**

মোট পড়তে হবে : ১৮টি

নিশ্চিত কমন : মিনিমাম ৩-৪ টি

নিশ্চিত মার্কস : ১৮-২৪ (প্রতি প্রশ্নে ৬ মার্কস)

**কিভাবে পড়বেন এই পাস বুক :****অধ্যায় বাছাই (সবার আগে শেষ করুন) :**

অধ্যায়	বিষয়	কারণ
১, ২	ডাটা স্ট্রাকচার ও অ্যালগরিদম	রচনামূলক কমন আসে
৪, ৫	স্ট্যাক ও কিউ	সংক্ষিপ্ত ও রচনামূলক কমন আসে
৬	লিংকড লিস্ট	গুরুত্বপূর্ণ ও বারবার আসে
৮, ৯	সার্চিং ও সর্টিং	গুরুত্বপূর্ণ ও বারবার আসে
৩, ৭	অ্যারে এবং ট্রি	পর্ব সমাপনী পরীক্ষায় আসে



স্টাডি প্ল্যান (মোট সময় : ৮ ঘণ্টা) :



২ ঘণ্টা → অধ্যায় ১ এবং ২



২ ঘণ্টা → অধ্যায় ৪ এবং ৫



৪ ঘণ্টা → অধ্যায় ৩, ৬-৯ পর্যন্ত



পাস বুকের প্রধান লক্ষ্য :



দ্রুত রিভিশনের সুবিধা- পরীক্ষার আগে সময় বাঁচায়



গুরুত্বপূর্ণ প্রশ্ন- উত্তরের সংক্ষিপ্ত বিন্যাস



সারাংশভিত্তিক সাজানো- এক নজরে পুরো বিষয়টা দেখে ফেলা



মূল টপিক ফোকাস- আত্মবিশ্বাস গড়ে তোলার সহায়ক



বিশেষ দ্রষ্টব্য :



এটি একটি পূর্ণাঙ্গ টেক্সটবুক নয়- বরং সঠিক সময় ব্যবস্থাপনার রিভিশন গাইড।



পরীক্ষার আগের “last moment” প্রস্তুতির জন্য উপযুক্ত সহকারী!



অধ্যায়-১

ডাটা স্ট্রাকচার

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। ডাটা কী?

BPSC- 12, বাকাশিবো- ০৫, ০৬, ০৮, ০৮'পরি, (১০-১৩), ১২'পরি, ১৯, ১৯'পরি

উত্তর : Data শব্দের আভিধানিক অর্থ তত্ত্ব বা “উপাত্ত”। “Datum” শব্দের বহুবচন “Data” হতে Data শব্দের উৎপত্তি। সহজ ভাষায়, কোনো কিছুই মান বা দলীয় মানকে ডাটা বা উপাত্ত বলে।

যেমন : Softmax, Rubel, 124019 ইত্যাদি।

*** ২। ইনফরমেশন বলতে কী বুঝায়?

BPSC- 2012, বাকাশিবো- ২০০৪, ০৫, ০৬, ০৮'পরি, ১২'পরি, ১৪, ১৪'পরি, ১৮, ২০

উত্তর : “Information” শব্দের আভিধানিক অর্থ “তথ্য”। Data কে Process বা প্রক্রিয়াকরণ করলে যে অর্থপূর্ণ অবস্থা পাওয়া যায়, তাকে ইনফরমেশন বলে।

যেমন : সে Softmax Online School এর কম্পিউটার বিভাগের ৪র্থ পর্বের শিক্ষার্থী।

*** ৩। Logical Data বা যৌক্তিক ডাটা কী?

উত্তর : একাধিক মানের মাঝে তুলনা করার জন্য যে সকল ডাটা ব্যবহৃত হয়, তাকে Logical বা যৌক্তিক ডাটা বলে। এই সকল ডাটার দুইটি মাত্র অবস্থা থাকে।

i) 1 = True বা সত্য বা Active বা High.

ii) 0 = False বা মিথ্যা বা Inactive বা Low.

**** ৪। String বা String Type Data বলতে কী বুঝ?**

বাকাশিবো-২০১৩' পরি, ১৮' পরি, ১৯, ২২

উত্তর : একাধিক Character এর সমষ্টিকে String বলে। যা অবশ্যই Double quotation (“ ”) এর মধ্যে লিখতে হয়।

যেমন : “Softmax Online School.”

***** ৫। Data Structure বলতে কী বুঝ?**

বাকাশিবো- ২০১০, ১১, ১১' পরি, ১২, ১৩' পরি, ১৬, ১৬' পরি, ১৭, ১৭' পরি, ১৮, ১৮' পরি, ১৯, ১৯' পরি, BRTA- 2012, BNPC- 2014, BCS- 36th

উত্তর : Logical or Mathematical model of a particular organization of data is called Data Structure. অর্থাৎ কোনো ডাটা সংগঠনের লজিক্যাল বা ম্যাথমেটিক্যাল মডেলকে Data Structure বলে।

যেমন : Array, Stack, Queue, Linked-List ইত্যাদি।

*** ৬। **Linear ও Non- Linear Data Structure** এর উদাহরণ দাও।

বাকশিবো- ২০০৯' পরি, ১৮' পরি, ১৯, ২১

উত্তর : Linear Data Structure : Array, Stack, Queue, Linked List ইত্যাদি।

Non-Linear Data Structure : Set, Graph, Tree ইত্যাদি।

*** ৭। **Data Structure** এর অপারেশনগুলি কী কী?

বাকশিবো- ২০০০, ০৭, ০৯, ০৯' পরি, ১০, ১০' পরি, ১১, ১২, ১৫

উত্তর : Data Structure এর অপারেশনগুলি নিম্নরূপ :

- | | |
|----------------|---------------|
| i) Traversing | ii) Inserting |
| iii) Searching | iv) Sorting |
| v) Deleting | vi) Merging |

SOS সংক্ষিপ্ত প্রশ্নোত্তর :

***১। **ডাটা ও ইনফরমেশনের মধ্যে পার্থক্য** লেখ।

বাকশিবো- ২০০৭, ০৯, ০৯' পরি, ১২, ১৩' পরি, ১৫, ১৬, ১৭, ১৭' পরি, ২০, ২২

উত্তর : ডাটা ও ইনফরমেশনের মধ্যে পার্থক্য নিম্নরূপ :

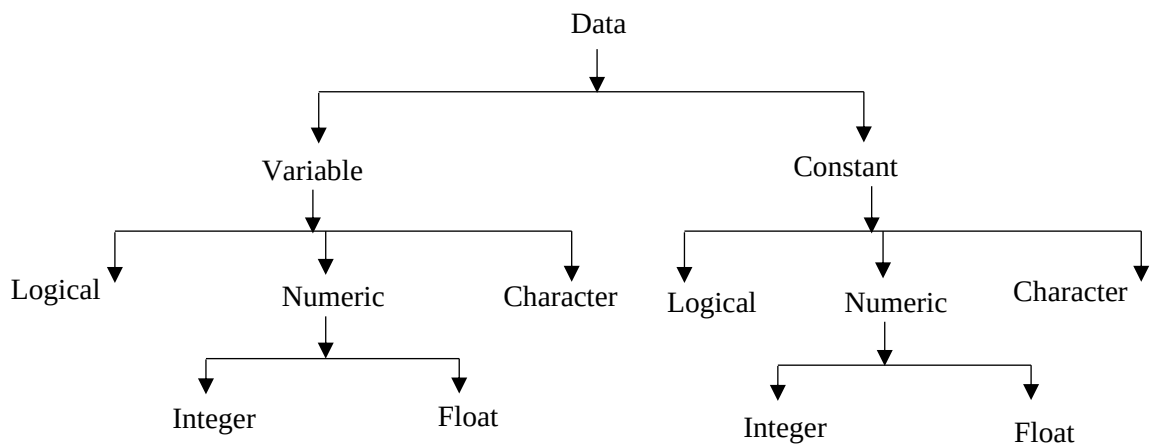
ডাটা	ইনফরমেশন
১) কোনো কিছুর মান বা দলীয় মানকে ডাটা বা উপাত্ত বলে।	১) Data কে Process বা প্রক্রিয়াকরণ করলে যে অর্থপূর্ণ অবস্থা পাওয়া যায়, তাকে ইনফরমেশন বলে।
২) Data শব্দের আভিধানিক অর্থ তত্ত্ব বা “উপাত্ত”।	২) “Information” শব্দের আভিধানিক অর্থ “তথ্য”।

৩) ডাটা কোনো পরিপূর্ণ অর্থ প্রকাশ করে না।	৩) ইনফরমেশন পরিপূর্ণ অর্থ প্রকাশ করে।
৪) ডাটা এলোমেলো অবস্থায় থাকে।	৪) ইনফরমেশন সাজানো অবস্থায় থাকে।
৫) তথ্যের ক্ষুদ্রতম এককই ডাটা।	৫) ডাটাকে প্রসেস করলে ইনফরমেশন পাওয়া যায়।
৬। ডাটা, প্রক্রিয়াকরণের পূর্বের অবস্থা।	৬) ইনফরমেশন, প্রক্রিয়াকরণের পরের অবস্থা।
৭) সকল ইনফরমেশনই ডাটা।	৭) সকল Data, ইনফরমেশন নয়।
৮) ডাটা, ইনফরমেশনের উপর নির্ভর করে না।	৮) ইনফরমেশন অবশ্যই ডাটার উপর নির্ভরশীল।

*** ২। ডাটা কত প্রকার ও কী কী?

বাকাশিবো- ২০০৫, ০৮, ০৯, ১২, ১৩, ১৩' পরি, ২০

উত্তর : ডাটার প্রকারভেদ নিম্নরূপ :



**** ৩। Data Structure কত প্রকার ও কী কী? উদাহরণসহ সংজ্ঞা দাও।**

বাকাশিবো- ২০০৬' পরি, ১৪' পরি,

DUET- 12-13, BCS- 36th, 37th, BNPC- 2014

উত্তর : Data Structure কে প্রধানত দুই ভাগে ভাগ করা হয়। যথা :

i) Linear Data Structure : যে ডাটা স্ট্রাকচারের উপাদানসমূহ মানের ক্রমানুসারে পর্যায়ক্রমিক সাজানো থাকে, তাকে Linear Data Structure বলে।

যেমন : Array, Stack, Queue, Linked-List ইত্যাদি।

ii) Non- Linear Data Structure : যে ডাটা স্ট্রাকচারের উপাদানসমূহ মানের ক্রমানুসারে পর্যায়ক্রমিক সাজানো থাকে না, তাকে Non-Linear Data Structure বলে।

যেমন : Set, Graph, Tree ইত্যাদি।

***** ৪। ডাটা স্ট্রাকচারের অপারেশনগুলো বর্ণনা কর।**

বাকাশিবো- ২০০৭, ১২, ১৩' পরি, ১৫, ১৫' পরি, ১৮, ১৮' পরি, ১৯' পরি, ২০

NTRCA- 2014, DUET: 2007-08, 13-14

উত্তর : ডাটা স্ট্রাকচারের অপারেশনগুলো নিম্নরূপ :

i) Traversing : কোনো Data Structure এ Data গুলোকে একবার Access এবং Process করাকে Traversing বলে।

ii) Inserting : পূর্বের তৈরিকৃত কোনো Data Structure এ নতুন Data যুক্ত করাকে Inserting বলে।

iii) Deleting : Data Structure হতে কোনো Data কে বাদ বা কেঁটে ফেলাকে Deleting বলে।

- iv) Searching :** কোনো নির্দিষ্ট Key value এর মাধ্যমে নির্দিষ্ট কোনো Data বা Location খুঁজে বের করাকে Searching বলে ।
- v) Sorting :** ডাটাসমূহকে মানের ক্রমানুসারে সাজানাকে Sorting বলে । Sorting দুই প্রকার । যথা :
- a) Ascending order :** ছোট থেকে বড় (A-Z, 0-9)
 - b) Descending order :** বড় থেকে ছোট (Z-A, 9-0)
- vi) Merging :** দুইটি ভিন্ন ফাইলকে একত্রিত করাকে Merging বলে ।

অধ্যায়-২

অ্যালগরিদম

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। Algorithm বলতে কী বুঝায়?

ICT Ministry- 2017,

বাকাশিবো- ২০০৯, ০৯' পরি, ১১, ১৩' পরি, ১৫' পরি, ১৮, ১৯, ২২

উত্তর : কোনো সমস্যা সমাধানের যুক্তিসম্মত ও পর্যায়ক্রমিক ধারার প্রক্রিয়াকে Algorithm বলে। Algorithm এর সর্ব প্রথম ধারণা দেন আরবি গণিতবিদ মুহাম্মদ ইবনে মুসা-আল-খোয়ারিজমী।

*** ২। Pseudo Code বা সুডো কোড বলতে কী বুঝায়?

বাকাশিবো- ০৩, ০৬, ০৬' পরি, ০৮, ০৯, ০৯' পরি, ১০, ১১, ১২, ১৪' পরি, ১৫' পরি, ১৮, ২০

উত্তর : Pseudo শব্দের অর্থ ছদ্ম বা সত্য নয় এমন। অনেক সময় প্রোগ্রাম লেখার আগে প্রোগ্রামের একটি খসড়া তৈরি করা হয়, যা দেখতে অনেকটা প্রোগ্রামিং কোডের মত কিন্তু এটি আসলে কোড নয়। প্রোগ্রামের ধরন ও কার্যাবলী তুলে ধরার জন্য কিছু সংখ্যক নির্দেশনার সমষ্টিকে Pseudo Code (সুডো কোড) বা Program Design Language (PDL) বলে।

*** ৩। Algorithm Notation কী?

বাকাশিবো- ২০০৬, ০৭, ০৮, ০৯, ১২, ১৫' পরি, ১৬' পরি, ১৮' পরি

উত্তর : Algorithm এ বিধিবদ্ধ কিছু সংকেত ব্যবহার করা হয়। Algorithm এর এই বিধিবদ্ধ সকল সংকেতকে Algorithm Notation বলে।

যেমন : Big-0 Notation (O-Notation)

Omega Notation (Ω – Notation)

Theta Notation (θ – Notation)

** ৪। Flow Chart বা প্রবাহ চিত্র কী?

বাকাশিবো- ২০০০, ০২

উত্তর : যে চিত্রের মাধ্যমে কোন প্রোগ্রাম কীভাবে কাজ করবে তার গতিধারা নির্ধারণ করা হয়, তাকে Flowchart বা প্রবাহচিত্র বলে।

অর্থাৎ Algorithm এর চিত্রভিত্তিক রূপকে Flowchart বলে।

** ৫। Bug ও Debugging কী?

বাকাশিবো- ২০০৬, ১১, ১৭

উত্তর : Bug: Program এর মধ্যে যত ধরনের ত্রুটি আছে, তাই Bug.

Debug: Source Code এর ত্রুটি সংশোধন করাই হচ্ছে Debug.

* ৬। Algorithm এর Complexity নির্ণয়ের বিবেচ্য বিষয়গুলো কী কী?

বাকাশিবো- ২০১৬, ২৩

উত্তর : Algorithm এর Complexity নির্ণয়ের বিবেচ্য বিষয় তিনটি।

যথা : i) Worst Case ii) Average Case iii) Best Case.

***** ৭। Structure Programming বলতে কী বুঝায়?**

বাকশিবো- ২০০৪, ০৭, ০৯, ০৯' পরি, ১৫' পরি, ২২

উত্তর : Structure Programming হলো Logical programming পদ্ধতি, যা Object Oriented Programming (OOP) এর পূর্বের Programming Structure. এই সকল প্রোগ্রাম Top-Down Design পদ্ধতিতে কাজ করে। এই সকল প্রোগ্রামিংকে Modular Programming ও বলা হয়।

যেমন : C, ALGOL, Pascal ইত্যাদি।

SOS**সংক্ষিপ্ত প্রশ্নোত্তর :******* ১। Algorithm এর বৈশিষ্ট্যগুলি লেখ।**

বাকশিবো- ২০০৬, ০৭, ১৪' পরি, ১৬, ১৮, ১৯, ১৯' পরি, ২৩

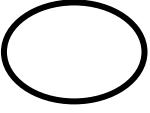
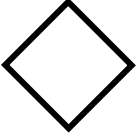
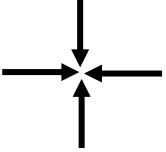
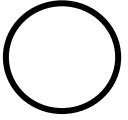
উত্তর : Algorithm এর বৈশিষ্ট্যগুলি নিম্নরূপ :

১. প্রতিটি Algorithm এর শুরু এবং শেষ থাকবে।
২. প্রতিটি Algorithm এর সুনির্দিষ্ট কতগুলো Step বা ধাপ থাকবে।
৩. প্রতিটি Algorithm এর এক বা একাধিক Input থাকবে।
৪. প্রতিটি Algorithm এর সুনির্দিষ্ট Output থাকবে।
৫. প্রতিটি ধাপ সহজবোধ্য হতে হবে।
৬. প্রতিটি ধাপের মাঝে পারস্পরিক সম্পর্ক থাকতে হবে এবং
৭. প্রতিটি ধাপের কাজ যুক্তি নির্ভর হতে হবে।

*** ২। **Flow-chart** এর বিভিন্ন প্রতীকগুলির নাম ও কাজ লেখ।

বাকাশিবো- ২০০৯' পরি, ১২, ১২' পরি, ১৫' পরি

উত্তর : Flow-chart এর বিভিন্ন প্রতীকগুলির নাম ও কাজ নিম্নরূপ :

প্রতীক	নাম	কাজ
	Start/End	Flowchart শুরু ও শেষ করার জন্য এই চিহ্ন ব্যবহৃত হয়।
	Input/Output	Input এবং Output নেওয়ার জন্য এই চিহ্ন ব্যবহার করা হয়।
	Process বা প্রক্রিয়াকরণ	গাণিতিক সকল Expression এ এই চিহ্ন ব্যবহার করা হয়।
	Decision Making বা সিদ্ধান্ত গ্রহণ	শর্তাধীন কোনো কাজ করার জন্য এই চিহ্ন ব্যবহার করা হয়।
	Flow বা নির্দেশনা	Program Flow কোন দিকে যাচ্ছে তা বুঝানোর জন্য এই চিহ্ন ব্যবহৃত হয়।
	Connection বা সংযোগ	একাধিক অংশকে সংযুক্ত করার জন্য এই চিহ্ন ব্যবহৃত হয়।
	Print বা Document	Program এর কোনো অংশ Print করার দরকার হলে এই চিহ্ন ব্যবহৃত হয়।
	Loop	একই কাজ বারবার (Looping) করার জন্য ব্যবহৃত হয়।

**** ৩। Algorithm ও Flowchart এর মাঝে পার্থক্য লেখ।**

বাকাশিবো- ২০০৯'পরি

উত্তর : Algorithm ও Flowchart এর মাঝে পার্থক্য নিম্নরূপ :

Algorithm	Flowchart
১) নির্দিষ্ট সমস্যা সমাধানের জন্য যুক্তিসম্মত ও ধাপে ধাপে সমাধানের পদ্ধতিকে Algorithm বলে।	১) Algorithm এর চিত্রভিত্তিক রূপকে Flowchart বলে।
২) এটি বর্ণনামূলক।	২) এটি চিত্রনির্ভর।
৩) Algorithm এর ভিত্তিতে Flowchart তৈরি করা হয়।	৩) Flowchart এর ভিত্তিতে Program রচনা করা হয়।
৪) এতে কোনো চিত্র ব্যবহৃত হয় না।	৪) এতে চিত্র ব্যবহৃত হয়।
৫) Algorithm তৈরির পূর্বে সুডো কোড তৈরির প্রয়োজন।	৫) Flowchart তৈরির পূর্বে Algorithm লেখা প্রয়োজন।

[**Note:** সুডো কোড : Program এর ধরন এবং কার্যপ্রণালী সংবলিত কিছু নির্দেশকেই সুডো বা Pseudo Code বলে।]

** ৪। ফারেনহাইট তাপমাত্রাকে সেন্টিগ্রেড এ রূপান্তরের Algorithm ও Flowchart আঁক।

বাকশিবো- ২০১৩' পরি, ১৭' পরি

উত্তর : ফারেনহাইট তাপমাত্রাকে সেন্টিগ্রেড এ রূপান্তরের Algorithm ও Flowchart নিম্নরূপ :

Algorithm

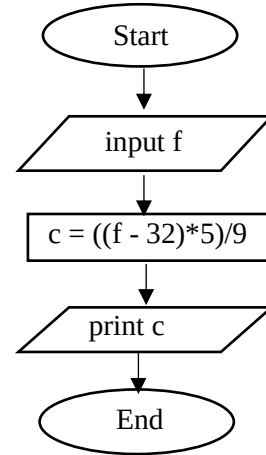
Step 1: Start

Step 2: input f

Step 3: $c = ((f - 32) * 5) / 9$

Step 4: print c

Step 5: End



Flowchart

তাপমাত্রার স্কেল

পাঠ নিম্ন – স্থিরাক্ষ

উর্ধ্ব স্থিরাক্ষ – নিম্ন স্থিরাক্ষ

$$\text{বা, } \frac{C-0}{100-0} = \frac{F-32}{212-32} = \frac{K-273}{373-273} = \frac{R-0}{80-0}$$

$$\text{বা, } \frac{C}{100} = \frac{F-32}{180} = \frac{K-273}{100} = \frac{R}{80}$$

$$\text{বা, } \frac{C}{5} = \frac{F-32}{9} = \frac{K-273}{5} = \frac{R}{4}$$

100	212	373	80
0	32	273	0
C	F	K	R

❖ Fibonacci Series ❖

Fibonacci Series : Fibonacci Series এর আবিষ্কারক ত্রয়োদশ শতাব্দীর বিখ্যাত গণিতবিদ ‘Leonardo Da Piso’ উনার ডাকনাম ছিল Fibonacci. এ সিরিজটি শুরু হয় 0 থেকে এবং সিরিজের পরবর্তী সংখ্যাগুলো প্রতিটি তার পূর্বের দুইটি পদের যোগফল।

যেমন : 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89.....

Fibonacci Series এর বৈশিষ্ট্য :

- i) যেকোনো সংখ্যা তার আগের দুটি রাশির যোগফলের সমান।
- ii) যেকোনো সংখ্যা তার পরের দুটি রাশির বিয়োগফলের সমান।
- iii) যেকোনো সংখ্যাকে তার আগের সংখ্যা দিয়ে ভাগ করলে Golden Ratio (1.618) পাওয়া যায়, (0, 1, 2) বাদে।

বাস্তব উদাহরণ :

- i) পাখির ঝাঁক বেধে আকাশে উড়া।
- ii) সূর্যমুখী ফুলের পাপড়ির বিন্যাস।
- iii) শামুকের প্যাচ।
- iv) ফুলকপির বিন্যাস।
- v) মৌমাছির পরিবার তন্ত্র।
- vi) মহাকাশের ছায়াপথ।
- vii) বেশিরভাগ ফুলের পাপড়ির বিন্যাস ইত্যাদি।

*** ৫। Fibonacci Series বের করার Algorithm লেখ।

বাকশিবো- ২০০৬, ০৬' পরি, ০৯, ১২' পরি, ১৬' পরি, ২২

উত্তর : Algorithm :

Step 1 : যদি $n=0$ অথবা $n=1$ হয়, তবে, $FIB=n$ এবং Return.

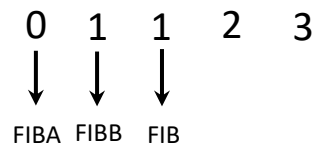
Step 2 : Call FIBONACCI (FIBA, N-1)

Step 3 : Call FIBONACCI (FIBB, N-2)

Step 4 : Set $FIB = FIB A + FIB B$

Step 5 : Return

ব্যাখ্যা :



[**Note:** পরীক্ষাতে উত্তর হিসেবে শুধু Algorithm লিখলেই হবে]

*** ৬। Factorial বের করার Algorithm লেখ।

বাকশিবো- ২০১৪' পরি, ১৭' পরি, ১৯

উত্তর : Factorial বের করার Algorithm নিম্নরূপ :

Algorithm :

Step 1 : যদি $N=0$ তবে, Set $FACT=1$ এবং Return করবে।

Step 2 : Call FACTORIAL (FACT, N-1)

Step 3 : Set $FACT = K * FACT$

Step 4 : Return.

ব্যাখ্যা : $n! = n(n-1)! = n(n-1)(n-2)! = n(n-1)(n-2) \dots \dots (n-n)!$

উদাহরণ : $4!$

$$= 4 \times 3!$$

$$3 \times 2!$$

$$2 \times 1!$$

$$1 \times 0!$$

$$1 \times 1$$

$$2 \times 1$$

$$3 \times 2$$

$$4 \times 6$$

*** ৭। Algorithm Complexity বলতে কী বুঝ?

বাকশিবো- ২০০৫, ০৯, ০৯' পরি, ১১, ১২, ১৪, ১৪' পরি, ১৫' পরি, ১৯' পরি

BCS- 35th, 36th, কারিগরি শিক্ষা অধিদপ্তর- ২০১৯

উত্তর : যখন কোনো Algorithm ডিজাইন করা হয়, তার আগে জানা প্রয়োজন Algorithm টি কতটা কার্যকর। সেক্ষেত্রে নিচের তিনটি বিষয় মাথায় রাখতে হবে :

- Algorithm টি কী নির্ধারিত সময়ের মধ্যে ফলাফল দিবে?
- সর্বোচ্চ কত বড় বা কতগুলি মানের জন্য এটি কাজ করবে?
- কতটুকু মেমরি ব্যবহার করবে?

Algorithm Complexity এর মাধ্যমে প্রথম দুটি বিষয়ের উত্তর পাওয়া যায়। একটি Algorithm যতগুলো Instruction ব্যবহার করে

কাজ করে তাই Algorithm এর Complexity. Complexity থেকে বুঝা যায় Algorithm টি তুলনামূলকভাবে কতটা ভালো।

Algorithm Complexity দুই ধরনের। যথা :

i) Time Complexity : একটি প্রোগ্রাম রান হতে কতক্ষণ সময় লাগে, তাকে Time Complexity বলে।

ii) Space Complexity : একটি প্রোগ্রাম মেমরিতে কতটুকু জায়গা নেয়, তাকে Space Complexity বলে।

Algorithm Complexity তিনটি বিষয়ের উপর নির্ভরশীল।

i) Worst Case : Running Time ও Space সবচেয়ে বেশি।

ii) Average Case : Running Time ও Space গড় মানের।

iii) Best Case : Running Time ও Space সবচেয়ে কম।

SOS**রচনামূলক প্রশ্নোত্তর :**

*** ১। তিনটি সংখ্যার মাঝে ক্ষুদ্রতম বা বৃহত্তম সংখ্যা নির্ণয়ের জন্য একটি অ্যালগরিদম ও ফ্লোচার্ট অঙ্কন কর।

বাকশিবো- ২০০৯, ০৯' পরি, ১৫' পরি, ১৬, ১৮, ১৮' পরি, ২২

উত্তর :**Algorithm**

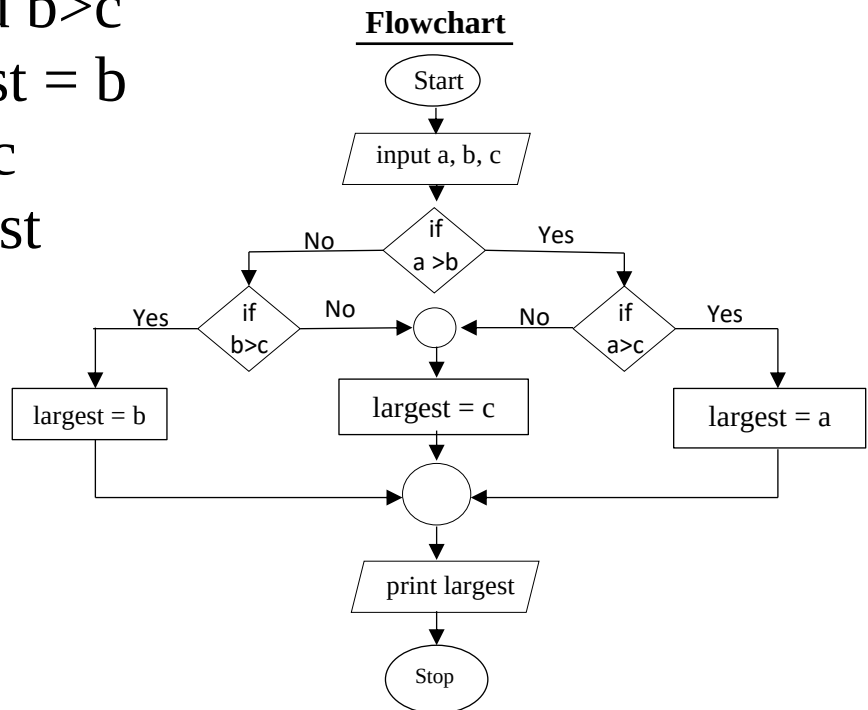
Step 1: Start

Step 2: input three numbers a, b, c

Step 3: if $a > b$ and $a > c$
 then largest = a
 else if $b > a$ and $b > c$
 then largest = b
 else largest = c

Step 4: print largest

Step 5: Stop



Note: ছোট সংখ্যা নির্ণয় করতে বললে Greater than ($>$) এর পরিবর্তে less Than ($<$)

*** ২। দ্বিঘাত সমীকরণ বা Quadratic equation ($ax^2 + bx + c = 0$) এর মূল বের করার Algorithm ও Flowchart

আঁক। বাকাশিবো- ২০০০, ০২, ০৫, ০৮, ১০, ১২, ১২' পরি, ১৫' পরি, ১৮' পরি, ১৯, ২২

উত্তর :

Algorithm

Step 1: Start

Step 2: input a, b, c

Step 3: নিশ্চায়ক $D = b^2 - 4ac$

Step 4: if $D > 0$ then

$$(i) x_1 = \frac{-b + \sqrt{D}}{2a}$$

$$(ii) x_2 = \frac{-b - \sqrt{D}}{2a}$$

print x_1, x_2

else if $D = 0$ then

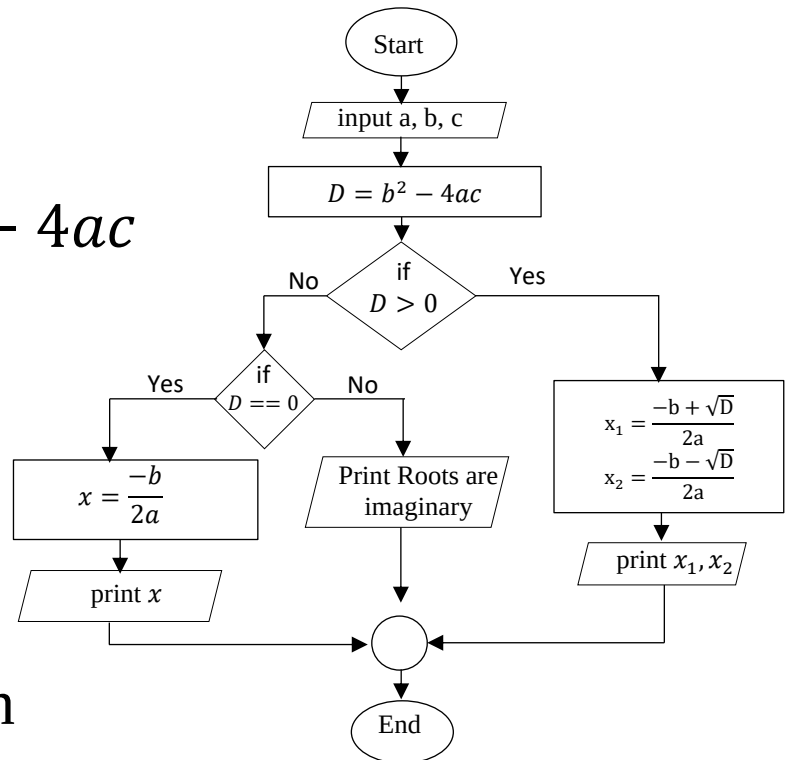
$$x = \frac{-b}{2a}$$

print x

else roots are imaginary

Step 5: End

Flowchart



*** ৩। ত্রিভুজের তিন বাহুর সাহায্যে ত্রিভুজের ক্ষেত্রফল নির্ণয়ের

Algorithm ও Flowchart আঁক।

উত্তর : শর্ত : যে কোনো দুই বাহুর যোগফল অবশ্যই তার তৃতীয় বাহু অপেক্ষা বড় হতে হবে। অন্যথায় ত্রিভুজ গঠন করা যাবে না।

অর্থাৎ, $(a + b) > c$ এবং $(b + c) > a$ এবং $(a + c) > b$ সবগুলি শর্তই সত্য হতে হবে।

Algorithm

Step 1: Start

Step 2: input a, b, c

Step 3: if $(a + b) > c$ and $(b + c) > a$ and $(a + c) > b$ then

(i) $S = (a + b + c)/2$

(ii) $area = \sqrt{S \times (S - a) \times (S - b) \times (S - c)}$

print area

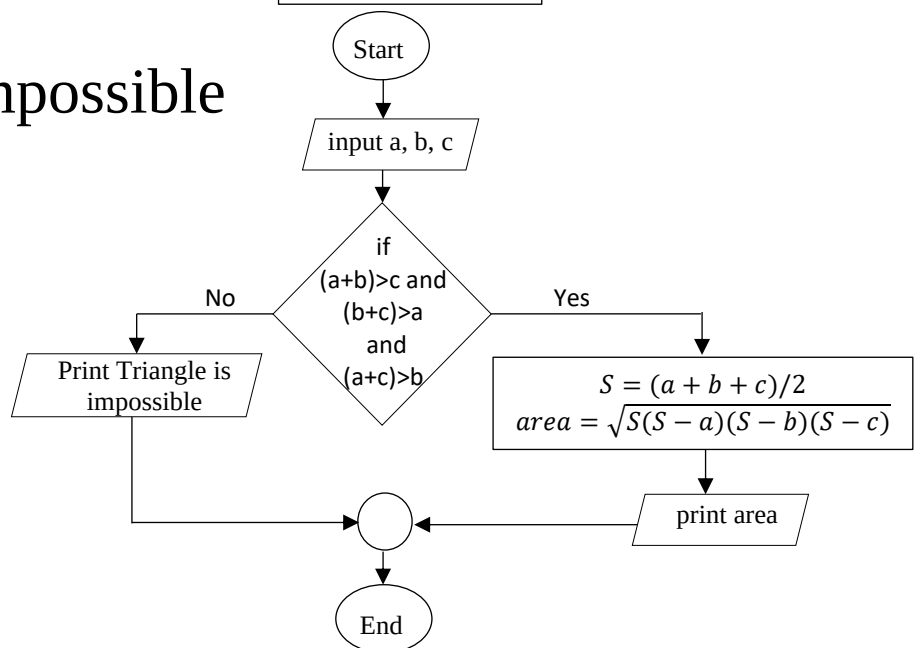
else

print Triangle is impossible

Step 4: Stop

$2S =$ পরিসীমা (সবগুলো বাহুর যোগফল)
 $S =$ অর্ধপরিসীমা

Flowchart



*** ৪। প্রোগ্রাম পরিকল্পনার ধাপসমূহ বর্ণনা কর।

বাকশিৰো- ২০০২, ১০, ১২' পরি, ১৩' পরি

উত্তর : প্রোগ্রাম দিয়ে নির্দিষ্ট কোনো সমস্যা সমাধান করতে হলে অবশ্যই সুনির্দিষ্ট কতগুলো ধাপে তা সম্পন্ন করতে হয়। ইহাকে প্রোগ্রাম পরিকল্পনার ধাপ বা Program Planning বলে। কারণ সুনির্দিষ্ট পরিকল্পনা ছাড়া কোনো কাজই সুসম্পন্ন হয় না।

প্রোগ্রাম পরিকল্পনার ধাপসমূহ নিম্নরূপ :

১) সমস্যা চিহ্নিত করা (**Problem Identification**) :- এটি প্রোগ্রাম পরিকল্পনার সর্বপ্রথম ধাপ। কারণ সবার প্রথমে আমি কি করতে চাই তা নির্ধারণ করতে হবে।

২) সমস্যা বিশ্লেষণ (**Problem Analysis**) : এই ধাপে সমস্যাকে বিশ্লেষণ করে সবচেয়ে সহজ উপায়ে সমস্যার সমাধান নির্ধারণ করা হয়।

৩) প্রোগ্রামের ইনপুট/আউটপুট নির্ণয় : এই ধাপে প্রয়োজনীয় Input ও Output নির্ধারণ করে কাক্সিত Output পাওয়ার জন্য সকল সূত্রের প্রয়োগ করতে হয়।

৪) প্রোগ্রামের অ্যালগরিদম উন্নয়ন করা (**Algorithm Development**) : Program টি কিভাবে কাজ করবে তা Algorithm ধাপে উল্লেখ করা থাকে। Program লেখার আগে তা কিভাবে কাজ করবে সেই সম্পর্কে অবশ্যই স্বচ্ছ ধারণা থাকতে হবে।

৫) ফ্লোচার্ট তৈরি করা (**Flowchart Development**) : Algorithm এর চিত্রভিত্তিক রূপই হলো Flowchart। এর জন্য নির্দিষ্ট কিছু চিত্র আছে, যা Program শুরু করার আগে Algorithm অনুযায়ী তৈরি করতে হবে।

৬) প্রোগ্রামিং ভাষা নির্ধারণ : অনেক প্রোগ্রামিং ভাষা থেকে আমার সমস্যার ধরন অনুযায়ী কোন ভাষা ব্যবহার করা বেশি সুবিধাজনক তা নির্ধারণ করতে হবে।

৭) প্রোগ্রাম রচনা করা (**Program Coding**) : এই ধাপে যে প্রোগ্রামিং ল্যাংগুয়েজ নির্ধারণ করা হয়েছে সেই অনুযায়ী Program লেখা শুরু করতে হবে।

৮) প্রোগ্রাম কম্পাইল করা : High Level Language কে Machine Language এ রূপান্তরের জন্য Compiler দ্বারা Program কে Compile করা হয়। ফলে যদি Program এ কোনো ভুল থাকে তবে তা ধরা পড়ে।

৯) প্রোগ্রাম পরীক্ষণ ও সংশোধন (**Program Testing And Debugging**) : Compile করার পর যে ভুলগুলি থাকে তাকে Bug বলে এবং এই Bug দূর করা বা সংশোধন করাকে Debugging বলে এবং বিভিন্ন ধরনের Testing (Alpha Test, Beta Test) ইত্যাদি করা হয়।

১০) **Documentation** : এই ধাপে সকল কার্যক্রমকে ভবিষ্যতের জন্য সংগ্রহ করে রাখা হয়। যেন পরবর্তীতে প্রয়োজন অনুযায়ী তা আবার ব্যবহার বা পরিবর্তন করা যায়।

১১) **Installation** : সকল Testing সম্পন্ন হবার পর প্রোগ্রামসমূহকে প্রোডাকশন লাইব্রেরীতে Load করা হয়। একে Installation বলে।

১২) **রক্ষণাবেক্ষণ (Maintenance)** : এটি প্রোগ্রাম পরিকল্পনার সর্বশেষ ধাপ। এতে ছোট কোনো পরিবর্তন বা পরিবর্ধন করা বুঝায়। কারণ একটি Software যখন বাজারে আসে তাতে নতুন কোনো আপডেট আসতেই পারে। ফলে তার উন্নত Version বাজারে আনতে হয়।

[**Note:** এই প্রশ্ন সংক্ষিপ্ত আকারে আসলে শুধু মাত্র ধাপ গুলো লিখলেই হবে]

***** ৫। Algorithm এর Complexity বর্ণনা কর।**

বাকশিবো- ২০০২, ০৬, ১৩'পরি, ১৫, ১৭'পরি, ১৮'পরি

উত্তর : **Algorithm :** কোনো সমস্যা সমাধানের যুক্তিসম্মত ও পর্যায়ক্রমিক ধারার প্রক্রিয়াকে Algorithm বলে। Algorithm এর সর্ব প্রথম ধারণা দেন আরবি গণিতবিদ মুহাম্মদ ইবনে মুসা-আল-খোয়ারিজমী।

Algorithm Complexity: যখন কোনো Algorithm ডিজাইন করা হয়, তার আগে জানা প্রয়োজন Algorithm টি কতটা কার্যকর।

সেক্ষেত্রে নিচের তিনটি বিষয় মাথায় রাখতে হবে :

- i) Algorithm টি কী নির্ধারিত সময়ের মধ্যে ফলাফল দিবে?
- ii) সর্বোচ্চ কত বড় বা কতগুলি মানের জন্য এটি কাজ করবে?
- iii) কতটুকু মেমরি ব্যবহার করবে?

Algorithm Complexity এর মাধ্যমে প্রথম দুটি বিষয়ের উত্তর পাওয়া যায়। একটি Algorithm যতগুলো Instruction ব্যবহার করে কাজ করে তাই Algorithm এর Complexity. এটি থেকে বুঝা যায় Algorithm টি তুলনামূলকভাবে কতটা ভালো।

Algorithm Complexity দুই ধরনের। যথা :

i) Time Complexity : একটি প্রোগ্রাম রান হতে কতক্ষণ সময় লাগে, তাকে Time Complexity বলে।

ii) Space Complexity : একটি প্রোগ্রাম মেমরিতে কতটুকু জায়গা নেয়, তাকে Space Complexity বলে।

Algorithm Complexity তিনটি বিষয়ের উপর নির্ভরশীল। যথা :

- i) Worst Case** : Running Time ও Space সবচেয়ে বেশি।
- ii) Average Case** : Running Time ও Space গড় মানের।
- iii) Best Case** : Running Time ও Space সবচেয়ে কম।

Algorithm এ বিধিবদ্ধ কিছু সংকেত ব্যবহার করা হয়। Algorithm এর এই বিধিবদ্ধ সকল সংকেতকে Algorithm Notation বলে।

যেমন : Big-0 Notation (O -Notation)

Omega Notation (Ω – Notation)

Theta Notation (θ – Notation)

অধ্যায়-৩

অ্যারে, পয়েন্টার এবং স্ট্রিং

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। লিনিয়ার অ্যারে (Linear Array) বলতে কী বুঝায়?

বাকাশিবো- ২০০২, ০৬

উত্তর : একই টাইপের একাধিক ভেরিয়েবলের সমষ্টিকে অ্যারে বলে। যেখানে ডাটাগুলো পরপর অবস্থান করে। ইহা Linear Data Structure, Array এর Indexing সবসময় 0 থেকে শুরু হয়।

Array ঘোষণার Syntax:

Data_Type Array_Name [Size];

i. e : int a [10];

** ২। One Dimensional Array কাকে বলে?

উত্তর : যে সকল অ্যারের একটিমাত্র Index বা Subscript থাকে, তাকে One Dimensional Array বা একমাত্রিক অ্যারে বলে।

Syntax:

Date_Type Array_Name [Size];

i. e : int a [10];

**** ৩। Two-Dimensional Array কী?**

উত্তর : যে সকল অ্যারের দুইটি Subscript থাকে, তাকে Two Dimensional (2D) বা দ্বিমাত্রিক অ্যারে বলে।

Syntax:

Date_Type Array_Name [row] [column];

i. e: int a [2] [3];

**** ৪। Array এর Length বের করার সূত্রটি লেখ।** বাকশির্বো- ২০১২

উত্তর : Array এর Length বা উপাদান সংখ্যা = $UB - LB + 1$

যেখানে, UB = Upper Bound (সর্বশেষ উপাদান)

LB = Lower Bound (প্রথম উপাদান)

**** ৫। Location নির্ণয়ের সূত্রটি লেখ, যা Linear Array এর K-তম উপাদান।** বাকশির্বো- ২০২৩

উত্তর : সূত্র : $Loc (LA[K]) = Base (LA) + W(K-LB)$

এখানে, LOC = Location

LA = Linear Array

K = যে উপাদানের Address বের করতে হবে। Base = Base Address বা Starting Address

W = Word Size (প্রতি উপাদানের জন্য মেমরিতে বরাদ্দকৃত জায়গা)

LB = Lower Bound

*** ৬। পয়েন্টার (Pointer) কাকে বলে?

বাকাশিবো- ২০০৭, ১৪'পরি

উত্তর : পয়েন্টার হলো এক বিশেষ ধরনের ভেরিয়েবল, যা নির্দিষ্ট টাইপের ভেরিয়েবলের অ্যাড্রেস ধারণ করে। পয়েন্টার ব্যবহার করে একজন প্রোগ্রামার সরাসরি Memory Address নিয়ে কাজ করতে পারেন।

Syntax:

Date_Type * Pointer_Name;

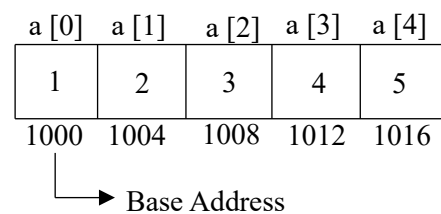
i. e : int * a;

*** ৭। Pointer Array কী?

বাকাশিবো- ২০০৫, ১৩' পরি, ১৭, ২১

উত্তর : Pointer কে যদি কোনো অ্যারের মাধ্যমে সজ্জিত করা হয়, তবে তাকে Pointer Array বলে।

যেমন : int a [5] = {1, 2, 3, 4, 5};
int&p;



ধরে নিলাম, Base Address = 1000

int এর জন্য Memory Size = 4 byte

** ৮। Jagged Array বলতে কী বুঝায়?

বাকাশিবো- ২০১২, ১৫'পরি, ১৯

উত্তর : Jagged Array এক বিশেষ ধরনের Two Dimensional Array, যেখানে Row এর সাইজ নির্দিষ্ট থাকবে কিন্তু প্রতিটি Row এর Column এর সাইজ প্রয়োজন অনুযায়ী কম বেশি করা যাবে। ইহাকে Ragged Array ও বলা হয়।

যেমন :

```
int [ ] [ ] a = new int [2] [ ];
a [0] = new int [3];
a [1] = new int [2]
```

00	01	02
10	11	

এখানে, ১ম Row তে 3টি কলাম এবং ২য় Row তে দুইটি কলাম।

*** ৯। Field, Record ও File কাকে বলে?

BPSC- 2014, বাকাশিবো- ২০০২, ০৪, ০৬, ০৮, ০৯, ১০, ১০'পর, ১২, ১২'পর, ১৩

উত্তর : **Field:** একই জাতীয় ডাটা উপাদানকে Field বলে।

যেমন : নাম, রোল, সেমিস্টার ইত্যাদি।

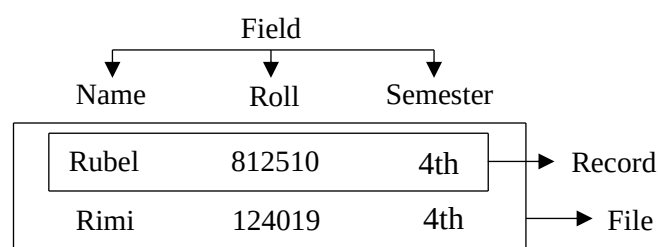
Record: একাধিক Field এর সমষ্টিকে Record বলে।

যেমন : একজন পরিক্ষার্থীর নাম, রোল এবং সেমিস্টার এই তিন Field এর সমন্বয়ে একটি Record গঠিত।

File: পরস্পর সম্পর্কযুক্ত একাধিক Record এর সমষ্টিকে File বলে।

যেমন : তোমার ক্লাসের সকল শিক্ষার্থীর নাম, রোল এবং সেমিস্টারের সকল Record নিয়ে একটি File গঠিত হয়।

উদাহরণ :



*** ১০। String কাকে বলে?

উত্তর : এক বা একাধিক Character এর সমষ্টিকে String বলে। যা double quotation (") এর মাঝে লিখতে হয়।

যেমন : `char str1 [ ] = "Softmax online school" ;`
সহজ ভাষায় String হচ্ছে Character Type এর Array.

SOS**সংক্ষিপ্ত প্রশ্নোত্তর :**

*** ১। Linear Array কে Memory তে উপস্থাপন করার কৌশল বর্ণনা কর।

বাকশির্ষো- ২০০২, ০৭, ০৯, ১১, ১২, ১২'পরি, ১৩, ১৩'পরি, ১৬'পরি

উত্তর : মনে করি, LA একটি Linear Array, যা কম্পিউটার মেমরিতে আছে।

আমরা জানি, কম্পিউটার মেমরিতে সংরক্ষিত অ্যারের উপাদানসমূহকে খুব সহজেই মেমরির পর্যায়ক্রমিক Address দ্বারা উপস্থাপন করা যায়। Array এর শুরুর Address অথবা Base Address জানা থাকলে ক্রমিক Address দ্বারা সকল উপাদানসমূহের Address বের করা যায়। Array এর যেকোনো Location এর Address নিচের সূত্র দ্বারা বের করা যায়।

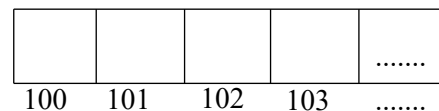
$$LOC(LA[K]) = Base(LA) + W(K-LB)$$

এখানে, LOC = Location

LA = Linear Array

K = যে উপাদানের Address বের করতে হবে।

Base = Base Address বা Starting Address



W = Word Size (প্রতি উপাদানের জন্য মেমরিতে বরাদ্দকৃত জায়গা)

LB = Lower Bound

Linear Array এর প্রতিটি উপাদানের Address কম্পিউটারের খুঁজে বের করার প্রয়োজন হয় না। Base Address জানা থাকলেই যে কোনো Location খুঁজে বের করা যায়।

একটি Array তে উপাদান সংখ্যা বের করার জন্য নিচের সূত্র ব্যবহার করা হয় :

Array এর উপাদান সংখ্যা = $UB - LB + 1$

*** ২। **Linear Array Traversing এর Algorithm লেখ।**

বাকশির্বো- ২০০০, ০৫, ০৭, ০৯, ১১, ১২, ১৩' পরি, ১৭' পরি, ১৮' পরি, ১৯, ২১, ২২

উত্তর : Algorithm:

Step 1: Start

Step 2: Initialize counter set $K = LB$

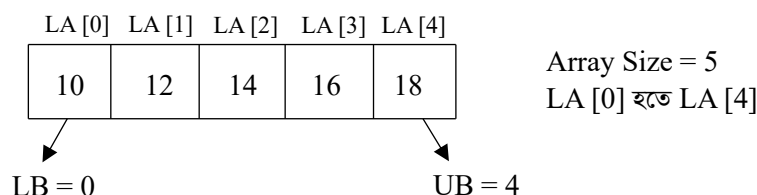
Step 3: Repeat steps 4 & 5 while $K \leq UB$

Step 4: Apply process to LA [K]

Step 5: Increment counter $K = K + 1$

Step 6: Stop

ব্যখ্যা :



প্রথমে Counter এর প্রারম্ভিক মান $K = LB = 0$ নির্ধারণ করে দেওয়া আছে। এরপর While loop ব্যবহার করে যতক্ষণ পর্যন্ত K এর মান UB এর চেয়ে ছোট বা সমান থাকবে ততক্ষণ পর্যন্ত পরবর্তী দুই ধাপ অর্থাৎ $LA[K]$ উপাদানের মান Access বা Traversing করতে এবং পরবর্তী Location এ যাওয়ার জন্য K এর মানের সাথে 1 যোগ করবে।

*** ৩। Linear Array তে Data Insert বা সংযোজনের Algorithm লেখ।

বাকশিবো- ২০০৬, ০৭, ০৯, ০৯'পরি, ১২, ১৬, ১৭'পরি, ১৮'পরি
প্রবাসী কল্যাণ ও বৈদেশিক কর্মসংস্থান জনশক্তি কর্মসংস্থান ব্যুরো- ২০১৮

উত্তর : Algorithm:

Step 1: Start

Step 2: Initialize counter set $J = N$

Step 3: Repeat Steps 4 & 5 while $J \geq K$

Step 4: Move Jth element downward

Set $LA[J + 1] = LA[J]$

Step 5: Decrease the counter, Set $J = J - 1$

Step 6: Insert element $LA[K] = \text{ITEM}$

Step 7: Update $N = N + 1$

Step 8: Stop

ব্যাখ্যা : এখানে $N = \text{Array size}$

$J = \text{Counter variable}$

$K = \text{যে location এ নতুন Data insert করব}$

$\text{ITEM} = \text{নতুন Data}$

LA	
A	0
B	1
C	2
E	3
F	4
G	5

$J = N = 6, K = 3$

LA	
A	0
B	1
C	2
	3
	4
	5
G	6

Counter এর মান N এ Initialize করতে হবে। একটি মানকে এক location নিচে বসিয়ে

Counter এর মান 1 বাড়াতে হবে। এভাবে 4 ও 5 নং Step চলবে। এরপর K তম অবস্থান

ITEM Insert করব। নতুন Data insert হওয়ায় Size 1 বেড়ে গেল। তাই $N = N + 1$ করা হয়েছে।

*** 8 | Linear Array হতে Data Delete করার Algorithm

লেখ।

বাকশিবো- ২০০৩, ০৬'পরি, ১২, ১৪'পরি, ১৯

উত্তর : Algorithm:

Step 1: Start

Step 2: Set ITEM = LA [K]

Step 3: Repeat for J = K to N-1

Move (J+1) element upward

Set LA [J] = LA [J+1]

(End of loop)

Step 4: Update $N = N - 1$

Step 5: Stop

LA	
A	0
B	1
C	2
D	3
E	4

ব্যাখ্যা : প্রথম কাজ হচ্ছে, যে উপাদানটি (ITEM) Delete করব, তাকে আগে খুঁজে বের করতে হবে। ধরি, 2 location এর মান Linear Array (LA) হতে একবার করে উপরে উঠবে। এভাবে পর্যায়ক্রমে Looping এর শর্ত পর্যন্ত চলবে। একটি Data delete হবার পর মোট Data হতে একটি Data কমাতে $n = n - 1$ করা হয়েছে।

*** ৫। **Matrix Multiplication (ম্যাট্রিক্স গুণন) এর Algorithm**

লেখ।

বাকশিবো- ২০০৬, ১০'পরি, ১২, ১৩'পরি, ১৪, ১৪'পরি, ১৬'পরি, ১৯, ২৩

উত্তর : Algorithm:Step 1: Repeat steps 2 and 3 for $I = 1$ to M Step 2: Repeat steps 3 and 4 for $J = 1$ to N Step 3: Set $C[I, J] = 0$ Step 4: Repeat for $K = 1$ to P

$$C[I, J] = C[I, J] + A[I, K] * B[K, J]$$

Step 5: Exit

ব্যাখ্যা : Matrix Multiplication (গুণনের) শর্ত :

প্রথম Matrix এর Column সংখ্যা, দ্বিতীয় Matrix এর Row এর সংখ্যা সমান হলে Matrix Multiplication করা যাবে।

$$\text{উদাহরণ : } A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}_{2 \times 2},$$

$$B = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \end{bmatrix}_{2 \times 3}$$

$$C = AB$$

C এর Order বা মাত্রা হবে $= 2 \times 3$

$$C_{11} = (A_{11} \times B_{11}) + (A_{12} \times B_{21})$$

$$C_{12} = (A_{11} \times B_{12}) + (A_{12} \times B_{22})$$

$$C_{13} = (A_{11} \times B_{13}) + (A_{12} \times B_{23})$$

$$\left| \begin{array}{l} A=M \times P \\ [M=I, P=K] \\ B=P \times N \\ [P=K, N=J] \\ C=M \times N \quad [I, J] \end{array} \right.$$

$$C_{21} = (A_{21} \times B_{11}) + (A_{22} \times B_{21})$$

$$C_{22} = (A_{21} \times B_{12}) + (A_{22} \times B_{22})$$

$$C_{23} = (A_{21} \times B_{13}) + (A_{22} \times B_{23})$$

$$C = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \end{bmatrix}$$

SOS**রচনামূলক প্রশ্নোত্তর :**

**** ১।** দ্বিমাত্রিক অ্যারেকে মেমরিতে উপস্থাপন পদ্ধতি বর্ণনা কর।

বাকশিবো- ২০০৭, ২৩

উত্তর : দ্বিমাত্রিক অ্যারে : যে সকল অ্যারের দুইটি Subscript থাকে, তাকে Two Dimensional (2D) বা দ্বিমাত্রিক অ্যারে বলে।

দ্বিমাত্রিক অ্যারেতে মেমরিতে উপস্থাপন : দ্বিমাত্রিক অ্যারেতে কম্পিউটার মেমরিতে দুই রকমভাবে উপস্থাপন করা যায়। যথা :

i) Row Major Order (রো ভিত্তিক) – Row $\times$ Row

ii) Column Major Order (সারি ভিত্তিক) – Column $\times$ Column

i) Row Major Order :

[0 0]	[0 1]	[0 2]	[0 3]	[1 0]	[1 1]	[1 2]	[1 3]	[2 0]	[2 1]	[2 2]	[2 3]
5	6	7	8	2	3	4	5	10	11	12	13
1 st Row				2 nd Row				3 rd Row			

মনে করি, 4 $\times$ 3 Size এর একটি দ্বিমাত্রিক অ্যারেতে নিম্নোক্ত Data গুলি আছে।

int a [3] [4];

	0	1	2	3
0	5	6	7	8
1	2	3	4	5
2	10	11	12	13

ii) Column Major Order :

[0 0]	[1 0]	[2 0]	[0 1]	[1 1]	[2 1]	[0 2]	[1 2]	[2 2]	[0 3]	[1 3]	[2 3]
5	2	10	6	3	11	7	4	12	8	5	13
1 st Column			2 nd Column			3 rd Column			4 th Column		

এখানে, M = Row size

N = Column size

BA = Base Address

J = যে Row এর মান বের করতে হবে।

K = যে Column এর মান বের করতে হবে।

W = Word size

নিচের দুইটি সূত্র ব্যবহার করে Row Major ও Column Major অনুসারে যেকোনো উপাদান বের করা যায়।

Row Major Order :

$$LOC(A[J,K]) = Base(A) + W [N (J-1) + (K-1)]$$

Column Major Order :

$$LOC(A[J,K]) = Base(A) + W [M (K-1) + (J-1)]$$

*** ২। Linear Array তে Data Insert বা সংযোজন এবং Data Delete বা বিয়োজনের Algorithm লেখ।

বাকশিবো- ২০১২, ১৩'পরি, ১৭, ১৭'পরি, ১৮

উত্তর : Linear Array তে Data Insert বা সংযোজনের Algorithm নিম্নরূপ :

Algorithm:

Step 1: Start

Step 2: Initialize counter set $J = N$

Step 3: Repeat Steps 4 & 5 while $J \geq K$

Step 4: Move Jth element downward

Set $LA [J + 1] = LA [J]$

Step 5: Decrease the counter, Set $J = J - 1$

Step 6: Insert element $LA [K] = ITEM$

Step 7: Update $N = N + 1$

Step 8: Stop

LA	
A	0
B	1
C	2
E	3
F	4
G	5

$J = N = 6, K = 3$

LA	
A	0
B	1
C	2
	3
	4
	5
G	6

ব্যাখ্যা : এখানে $N = \text{Array size}$

$J = \text{Counter variable}$

$K = \text{যে location এ নতুন Data insert করব}$

$ITEM = \text{নতুন Data}$

Counter এর মান N এ Initialize করতে হবে। একটি মানকে এক location নিচে বসিয়ে

Counter এর মান 1 বাড়াতে হবে। এভাবে 4 ও 5 নং Step চলবে। এরপর K তম অবস্থানে ITEM Insert করব। নতুন Data insert হওয়ায় Size 1 বেড়ে গেল। তাই $N = N + 1$ করা হয়েছে।

Linear Array তে Data Delete বা বিয়োজনের Algorithm নিম্নরূপ :

Algorithm:

Step 1: Start

Step 2: Set ITEM = LA [K]

Step 3: Repeat for J = K to N-1

Move (J+1) element upward

Set LA [J] = LA [J+1]

(End of loop)

Step 4: Update $N = N - 1$

Step 5: Stop

LA	
A	0
B	1
C	2
D	3
E	4

ব্যাখ্যা : প্রথম কাজ হচ্ছে, যে উপাদানটি (ITEM) Delete করব তাকে আগে খুঁজে বের করতে হবে। ধরি, 2 location এর মান Linear Array (LA) হতে একবার করে উপরে উঠবে। এভাবে পর্যায়ক্রমে Looping এর শর্ত পর্যন্ত চলবে। একটি Data delete হবার পর মোট Data হতে একটি Data কমাতে $N = N - 1$ করা হয়েছে।

অধ্যায়-৪

স্ট্যাক ডাটা-স্ট্রাকচার

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। Stack কী?

বাকাশিবো- ২০০৮, ০৮' পরি, ২০,

Bcs-36th, DUET: 2005-06, 07-08, 17-18

উত্তর : Stack এক ধরনের Linear Data Structure. যেখানে Data গুলি LIFO (Last in First Out) পদ্ধতিতে কাজ করে এবং Stack এ কেবলমাত্র End হতেই Data সংযোজন (Insert) এবং বিয়োজন (Delete) করা যায়।

যেমন : বইয়ের স্তুপ, ময়লার স্তুপ ইত্যাদি Stack এর বাস্তব উদাহরণ।

*** ২। PUSH এবং POP বলতে কী বুঝ?

বাকাশিবো- ২০০২, ০৩, ০৬, ০৬' পরি, ০৯, ১১, ১১' পরি, ১৬' পরি, ১৮, ১৯' পরি

উত্তর : **PUSH :** Stack এ নতুন কোনো Item সংযোজন বা Insert করার প্রক্রিয়াকে PUSH বলে। PUSH করলে TOP এর মান 1 করে বাড়ে।

POP : Stack হতে কোনো Item বিয়োজন বা Delete করার প্রক্রিয়াকে POP বলে। POP করলে TOP এর মান 1 করে কমে।

Note : TOP : Stack এর সর্বশেষ উপাদানের Address ধারণ করে। PUSH ও POP operation, TOP হতেই হয়।

*** ৩। LIFO বলতে কী বুঝায়?

বাকাশির্বো- ২০০৫, ১৬, ১৯' পরি

উত্তর : LIFO এর পূর্ণরূপ Last In First Out. অর্থাৎ যে Data Element সবার শেষে Insert হবে, সেই Data Element কে সবার প্রথমে Delete করতে হবে। Stack Data Structure এ LIFO ব্যবহৃত হয়।

*** ৪। Stack এর Overflow এবং Underflow কী?

বাকাশির্বো- ২০০৩, ০৪, ০৬, ০৭, ০৯, (১৩-১৪)'পরি, ১৬'পরি, (১৭-২০), (১৮-১৯)'পরি, ২৩

উত্তর : **Overflow :** Stack এ নতুন কোনো Data Insert করার প্রয়োজনীয় জায়গা না থাকলে Overflow সংগঠিত হয়।

Underflow : Stack হতে কোনো Data Delete করতে চাইলে যদি Stack এ কোনো Data না থাকে তাহলে Underflow সংগঠিত হয়।

ব্যাখ্যা : TOP = MAXSTK হলে Stack full, তখন কোনো কিছু Insert করতে চাইলে Overflow হয়।

TOP = 0 হলে Stack খালি, তখন কোনো কিছু Delete করতে চাইলে Underflow হয়।

TOP	30	5=MAXSTK
	20	4
	10	3
	2	2
	1	1

Polish Notation

Poland এর বিখ্যাত গণিতবিদ 'Jan Lukasiewicz' ১৮৯৫ সালে সর্ব প্রথম Polish Notation প্রবর্তন করেন। গণিতে ব্যবহৃত Notation পদ্ধতিগুলির মধ্যে এটি একটি বিশেষ ধরনের পদ্ধতি।

পরবর্তীতে এ পদ্ধতি নিয়ে বিভিন্ন গবেষণা হয় এবং বিভিন্ন Algorithm এর উন্নয়ন হয়।

Operator এর অবস্থানের উপর ভিত্তি করে ইহা তিন প্রকার। যথা :

- i) Infix Notation ($A+B$)
- ii) Prefix Notation ($+AB$)
- iii) Postfix Notation ($AB+$)

*** ৫। Infix Notation কী?

বাকশিবো- ২০০৫, ০৮, ১২, ১৮, ২২

উত্তর : যে গাণিতিক Notation পদ্ধতিতে Operand এর মাঝে Operator অবস্থান করে, তাকে Infix Notation বলে।

যেমন : $A+B$ এখানে, $A, B = \text{Operand}$
 $+ = \text{Operator}$

*** ৬। Prefix Notation বা Polish Notation কী?

বাকশিবো- ২০০০, ০৪, ০৫, ০৭, ০৮, ০৯, ০৯' পরি, ১৩' পরি, ১৫' পরি, ১৮' পরি, ১৯

উত্তর : যে গাণিতিক Notation পদ্ধতিতে Operand এর পূর্বে Operator অবস্থান করে, তাকে Prefix বা Polish Notation বলে।

যেমন : $A+B$ এর Prefix Notation $+AB$

*** ৭। **Postfix Notation বা Reverse Polish Notation**

বলতে কী বুঝায়? বাকশিবো- ০০, ০৫, ০৬' পরি, ০৮, ০৯, ১০, ১০' পরি, ১৪' পরি, ১৬

উত্তর : যে গাণিতিক Notation পদ্ধতিতে Operand এর পরে Operator অবস্থান করে, তাকে Postfix বা Reverse Polish Notation বলে। যেমন : $A+B$ এর Postfix Notation $AB+$

*** ৮। **$(A+B)*C$ এর Postfix Notation লেখ।**

বাকশিবো- ২০১১'পরি, ১৭, ২০

উত্তর : $(A+B)*C = [AB+]*C$
 $= AB+C*$

*** ৯। **$(x+y)*z$ এর Postfix Notation লেখ।**

বাকশিবো- ২০০০৯, ০৯'পরি, ১৭'পরি

উত্তর : $(x+y)*z = [xy+]*z$
 $= xy+z*$

** ১০। **Postfix-এ রূপান্তর কর :**

i) $(C-D)*C$

বাকশিবো- ২০২২

ii) $(A-B)* (C/D)$

বাকশিবো- ২০১৬' পরি

উত্তর : i) $(C-D)*C$
 $= [CD -] *C$
 $= CD - C*$

$$\begin{aligned}
 \text{ii) } (A-B) * (C/D) \\
 &= [AB-] * [CD /] \\
 &= AB - CD /*
 \end{aligned}$$

SOS**সংক্ষিপ্ত প্রশ্নোত্তর :**

*** ১। PUSH ও POP এর মাঝে পার্থক্য লেখ।

বাকাশিবো- ২০০৯'পরী, ১২, ১৫'পরী, ১৭'পরী, ১৯'পরী

উত্তর : PUSH ও POP এর মাঝে পার্থক্য নিম্নরূপ :

PUSH	POP
i) Stack এ নতুন কোনো Item সংযোজন বা Insert করার প্রক্রিয়াকে PUSH বলে।	i) Stack হতে কোনো Item বিয়োজন বা Delete করার প্রক্রিয়াকে POP বলে।
ii) PUSH Operation এ TOP এর মান 1 করে বাড়ে।	ii) POP Operation এ TOP এর মান 1 করে কমে।
iii) ইহা Data সংযোজনের এক বিশেষ প্রক্রিয়া।	iii) ইহা Data বিয়োজনের এক বিশেষ প্রক্রিয়া।
iv) TOP = MAXSTK থাকা অবস্থায় PUSH Operation করতে চাইলে Overflow হয়।	iv) TOP = 0 থাকা অবস্থায় POP Operation করতে চাইলে Underflow হয়।

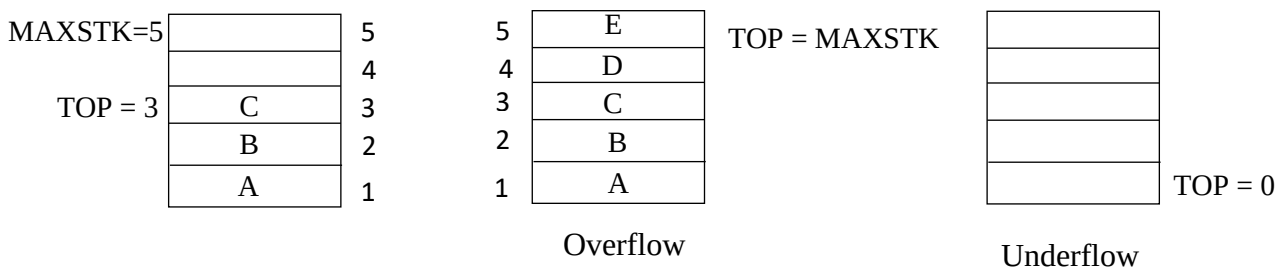
*** ২। STACK এর TOP, MAXSTK, Overflow এবং Underflow কী? উদাহরণসহ লেখ। বাকাশিৰো- ২০১১, ১১'পরি, ১৮'পরি, ১৯

উত্তর : TOP : Stack এর সর্বশেষ উপাদানের অবস্থান জানা যায় TOP এর মান দ্বারা। Data Insert করলে TOP এর মান 1 করে বাড়ে এবং Data Delete করলে TOP এর মান 1 করে কমে।

MAXSTK : Stack এর সর্বোচ্চ size কে MAXSTK বলা হয়।

Overflow : Stack এ নতুন কোনো Data Insert করার প্রয়োজনীয় জায়গা না থাকলে Overflow সংগঠিত হয়।

Underflow : Stack হতে কোনো Data Delete করতে চাইলে যদি Stack এ কোনো Data না থাকে তাহলে Underflow সংগঠিত হয়।



***** ৩। Stack এ Data সংযোজন বা PUSH করার Algorithm**

লেখ।

DUET: 2012-13, বাকাশিবো- ২০০৬' পরি, ০৮, ০৮' পরি, ০৯, ১০, ১১, ১৩' পরি, ১৪' পরি, ১৭, ১৯, ২০, ২৩**উত্তর : Algorithm :**

Step 1 : Start

Step 2 : if $TOP = MAXSTK$ then
print overflow and Return.Step 3 : Set $TOP = TOP + 1$ Step 4 : Set $STACK [TOP] = ITEM$

Step 5 : Stop

ব্যাখ্যা : প্রথমেই TOP এর মান MAXSTK এর সমান কী না, তা চেক করতে হবে। যদি মান সমান হয় তবে Overflow print হবে, অর্থাৎ Stack পরিপূর্ণ। আর যদি সমান না হয়, অর্থাৎ Stack এ জায়গা খালি থাকে তবে TOP এর মান 1 বাড়বে এবং TOP এর বৃদ্ধিকৃত মানের নতুন Data ITEM insert হবে।

*** 8 | Stack হতে Data বিয়োজন বা POP করার Algorithm

লেখ।

বাকশিবো- ২০০০, ০৫, ০৬, ০৮, ১০, ১০' পরি, ১১' পরি,
১২, ১৩' পরি, ১৫' পরি, ১৭, ১৮' পরি, ২২

উত্তর : Algorithm :

Step 1 : Start

Step 2 : if TOP = 0 then print underflow and Return

Step 3 : Set ITEM = STACK [TOP]

Step 4 : Set TOP = TOP – 1

Step 5 : Stop

ব্যাখ্যা : প্রথমেই TOP এর মান 0 (শূন্য) কী না তা চেক করতে হবে।
যদি 0 হয় তবে Underflow হবে অর্থাৎ Stack সম্পূর্ণ খালি, যাতে
Delete করার মত কোনো Data নেই। যদি 0 না হয়, তবে Stack এর
TOP এ যে Data আছে, তাকে Delete করতে হবে এবং TOP এর
মান 1 কমবে।

* ৫। $10 * (12 + 4) - 24 / 8$ কে Postfix expression এ রূপান্তর কর।

বাকশিবো- ২০২২

উত্তর :

$$\begin{aligned}
 & 10 * (12 + 4) - 24 / 8 \\
 &= 10 * [12 \ 4 \ +] - 24 / 8 \\
 &= 10 * [12 \ 4 \] - [24 \ 8 \ / \] \\
 &= [10 \ 12 \ 4 \ + \ *] - [24 \ 8 \ / \] \\
 &= 10 \ 12 \ 4 \ + \ * \ 24 \ 8 \ / \ - \quad \textbf{(Ans.)}
 \end{aligned}$$

*** ৬। P : 12, 7, 3, -, /, 2, 1, 5, +, *, + এর মান বের কর।

বাকশিবো- ২০০৫, ০৭, ১০, ১৭

উত্তর :

$$\begin{aligned}
 & P = 12, 7, 3, -, /, 2, 1, 5, +, *, + \\
 &= 12, [7 - 3], /, 2, 1, 5, +, *, + \\
 &= 12, 4, /, 2, 1, 5, +, *, + \quad [7-3=4] \\
 &= [12 / 4], 2, 1, 5, +, *, + \\
 &= 3, 2, 1, 5, +, *, + \quad [12/4=3] \\
 &= 3, 2 [1 + 5], *, + \\
 &= 3, 2, 6, *, + \quad [1+5=6] \\
 &= 3, [2 * 6], + \\
 &= 3, 12, + \quad [2 \times 6 = 12] \\
 &= [3 + 12] \\
 &= 15 \textbf{(Ans.)} \quad [3+12=15]
 \end{aligned}$$

SOS**রচনামূলক প্রশ্নোত্তর :**

*** ১। **STACK** এ উপাদান সংযোজনের পদ্ধতি বর্ণনা কর।

BCS- 36th, বাকাশিবো- ২০০৭, ০৯

উত্তর : চিত্রে একটি **STACK** দেখানো আছে। যার সর্বোচ্চ উপাদান সংখ্যা 5, আমরা **MAXSTK** এর মান দ্বারা বুঝতে পারি। **TOP** এর কাজ হচ্ছে **STACK** এর সর্বশেষ উপাদানের Address ধারণ করা। এখানে **TOP = 3**, অর্থাৎ এই **STACK** এ তিনটি উপাদান আছে। **STACK** এ Data

MAXSTK		5
		4
TOP	C	3
	B	2
	A	1

সংযোজনের সময় **TOP = MAXSTK** কিনা তা চেক করতে হবে। যদি তা হয় তাহলে Stack full বা পরিপূর্ণ। এই অবস্থায় কোনো কিছু Insert করতে চাইলে Overflow হবে। যদি তা না হয়, তবে **TOP** এর পরবর্তী অবস্থানে নতুন Data বা ITEM সংযোজন হবে এবং **TOP** এর মান 1 করে বাড়তে থাকবে।

Algorithm :

Step 1 : Start

Step 2 : if **TOP = MAXSTK** then
print overflow and Return.

Step 3 : Set **TOP = TOP + 1**

Step 4 : Set **STACK [TOP] = ITEM**

Step 5 : Stop

*** ২। STACK হতে উপাদান বিয়োজনের পদ্ধতি বর্ণনা কর।

BCS- 36th, বাকাশিবো- ২০০৯, ১১, ১৬' পরি

উত্তর : চিত্রে একটি STACK দেখানো আছে। যার সর্বোচ্চ উপাদান সংখ্যা 5, আমরা MAXSTK এর মান দ্বারা বুঝতে পারি। TOP এর কাজ হচ্ছে STACK এর সর্বশেষ উপাদানের Address ধারণ করা। এখানে $TOP = 3$, অর্থাৎ এই STACK এ তিনটি উপাদান আছে।

MAXSTK		5
		4
TOP	C	3
	B	2
	A	1

STACK এ Data বিয়োজনের সময় $TOP = 0$ কিনা তা চেক করতে হবে। যদি তা হয় তাহলে Stack empty বা খালি।

এই অবস্থায় কোনো কিছু Delete করতে চাইলে Underflow হবে। যদি তা না হয়, তবে TOP মান 1 করে কমতে থাকবে।

Algorithm :

Step 1 : Start

Step 2 : if $TOP = 0$ then print underflow and Return

Step 3 : Set $ITEM = STACK [TOP]$

Step 4 : Set $TOP = TOP - 1$

Step 5 : Stop

ব্যাখ্যা : প্রথমেই TOP এর মান 0 (শূন্য) কী না তা চেক করতে হবে। যদি 0 হয় তবে Underflow হবে অর্থাৎ Stack সম্পূর্ণ খালি, যাতে Delete করার মত কোনো Data নেই। যদি 0 না হয়, তবে Stack এর TOP এ যে Data আছে, তাকে Delete করতে হবে এবং TOP এর মান 1 কমবে।

*** ৩। নিম্নলিখিত Infix expression কে সমতুল্য Postfix expression এ রূপান্তর কর :

i) $(A/B) / ((D+E) * F)$

বাকাশিবো- ২০১৩' পরি

ii) $((A-B) / D \uparrow ((E+F) * G))$

বাকাশিবো- ২০০২

iii) $A + (B+D) / (E-F * (G + H / K))$

বাকাশিবো-০২, ১৬, ১৯' পরি

iv) $A + (B * C - (D / E \uparrow F) * G) * H$

BCS- 36th, বাকাশিবো- ২০০৩, ০৭

v) $(A \uparrow B + C/C) * (C - D)$

বাকাশিবো- ২০০৮

vi) $A \uparrow B * C - D + E * F / (G * H)$

বাকাশিবো- ০৮, ১০, ১০'পরি

উত্তর : i) $(A/B) / ((D+E) * F)$

$$= [AB /] / ([DE +] * F)$$

$$= [AB /] / [DE + F *]$$

$$= AB / DE + F * / (\text{Ans.})$$

ii) $((A-B) / D \uparrow ((E+F) * G))$

$$= ([AB -] / D \uparrow ([EF +] * G))$$

$$= [AB - D /] \uparrow [EF + G *]$$

$$= AB - D / EF + G * \uparrow (\text{Ans.})$$

i) $A + (B+D) / (E-F * (G + H / K))$

$$= A + [BD +] / (E - F * (G + [HK /]))$$

[+ এর চেয়ে / এর Precedence বেশি]

$$= A + [BD +] / (E - F * [GHK / +])$$

$$\begin{aligned}
&= A + [BD +] / (E - [FGHK / + *]) \\
&\quad [- \text{এর চেয়ে } * \text{ এর Precedence বেশি}] \\
&= A + [BD +] / [EFGHK / + * -] \\
&= A + [BD + EFGHK / + * - /] \\
&\quad [+ \text{এর চেয়ে } / \text{ এর Precedence বেশি}] \\
&= ABD + EFGHK / + * - / + \text{ (Ans.)}
\end{aligned}$$

$$\begin{aligned}
\text{ii) } &A + (B * C - (D / E \uparrow F) * G) * H \\
&= A + (B * C - (D / [E F \uparrow]) * G) * H \\
&\quad [/ \text{ এর চেয়ে } \uparrow \text{ এর Precedence বেশি}] \\
&= A + (B * C - [DEF \uparrow /]) * G) * H \\
&= A + ([BC *] - [DEF \uparrow / G *]) * H \\
&= A + [BC * DEF \uparrow / G * -] * H \\
&= A + [BC * DEF \uparrow / G * - H *] \\
&\quad [+ \text{ এর চেয়ে } * \text{ এর Precedence বেশি}] \\
&= ABC * DEF \uparrow / G * - H * + \text{ (Ans.)}
\end{aligned}$$

$$\begin{aligned}
\text{iii) } &(A \uparrow B + C/C) * (C - D) \\
&= ([AB \uparrow] + [CC /]) * (C - D) \\
&\quad [+ \text{ এর চেয়ে } \uparrow \text{ ও } / \text{ এর Precedence বেশি}] \\
&= [AB \uparrow CC / +] * [CD -] \\
&\quad [() \text{ এর Precedence সবচেয়ে বেশি}] \\
&= AB \uparrow CC / + CD - * \text{ (Ans.)}
\end{aligned}$$

$$\begin{aligned}
 \text{iv) } & A \uparrow B * C - D + E * F / (G * H) \\
 &= [AB \uparrow] * C - D + E * F / [GH *] \\
 &= [AB \uparrow C *] - D + [EF *] / [GH *] \\
 &= [AB \uparrow C *] - D + [EF * GH * /] \\
 &= [AB \uparrow C * D -] + [EF * GH * /] \\
 &= AB \uparrow C * D - EF * GH * / + \quad (\text{Ans.})
 \end{aligned}$$

Operator Precedence

Note: Infix হতে Postfix অথবা Infix হতে Prefix এ রূপান্তরের জন্য Operator Precedence অনুযায়ী Operation করতে হবে।

Operator Precedence:

Priority	Operator
1	()
2	↑
3	*, /
4	+, -

অধ্যায়-৫

কিউ ডাটা-স্ট্রাকচার

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

***** ১। Queue (কিউ) বলতে কী বুঝায়?**

DUET: 2005-06, 07-08; বাকাশিবো- ২০০৮, ০৮'পরী, ১৭, ১৭'পরী, ১৮

উত্তর : Queue এক ধরনের Linear Data Structure, যেখানে Data গুলি FIFO (First In First Out) পদ্ধতিতে Insert ও Delete করা হয়। অর্থাৎ যে Data প্রথমে Insert হবে সেই Data সবার আগে Delete হবে।

***** ২। DE-Queue বলতে কী বুঝায়?**

বাকাশিবো- ২০১২, ১৩'পরী, ১৭, ১৭'পরী, ২০, ২৩

উত্তর : DE-queue এর পূর্ণরূপ Double Ended Queue. এটি এক ধরনের Queue, যেখানে উপাদানগুলি তালিকার প্রথমে বা শেষে Insert বা Delete হতে পারে। কিন্তু মাঝখান হতে Insert বা Delete করা যাবে না।

Note: Dequeue: Queue হতে কোনো উপাদান Delete বা বিয়োজন করাকে Dequeue বলে।

***৩। Priority Queue কী? BCS- 36th, বাকাশিবো- ০৫, ১৮'পরি, ১৯'পরি

উত্তর : Priority Queue এক বিশেষ ধরনের Queue, যার প্রত্যেকটি Data উপাদানের জন্য Priority সেট করা থাকে এবং তা নিম্নোক্তভাবে কাজ করে।

i) Low Priority বিশিষ্ট উপাদানের আগে High Priority উপাদানের কাজ করতে হবে।

ii) সমান Priority বিশিষ্ট দুটি উপাদানের মধ্যে যে উপাদান আগে যুক্ত হয়েছে, তাকে আগেই Access প্রদান করতে হবে।

SOS**সংক্ষিপ্ত প্রশ্নোত্তর :**

*** ১। Queue তে উপাদান সংযোজনের Algorithm লেখ।

BCS-36th, বাকাশিবো- ২০০৩, ০৪, ০৯, ০৯'পরি, (১১-১৩), ১১'পরি, ১৩'পরি, (১৫-১৮), ১৬'পরি

উত্তর : **Algorithm:**

Step 1 : If FRONT = 1 and REAR = N

Or if FRONT = REAR + 1

then print Overflow and Return

Step 2 : If FRONT = NULL then

Set FRONT = REAR = 1

else if REAR = N then

Set REAR = 1

else

set REAR = REAR + 1

Step 3 : Set QUEUE [REAR] = ITEM

Step 4 : Return

ব্যাখ্যা :

1. যদি $FRONT = 1$ এবং $REAR = N$ হয় অথবা $FRONT = REAR + 1$ হয় (যা বাস্তবে সম্ভব নয়), তবে Queue Full বা পরিপূর্ণ। এই অবস্থায় নতুন কোনো ITEM সংযোজন করতে চাইলে Overflow হবে।

2. যদি Queue তে ITEM সংযোজন করার জন্য প্রয়োজনীয় খালি জায়গা থাকে, তবে-

i) যদি $FRONT = NULL$ হয়, মানে Queue সম্পূর্ণ খালি। এই অবস্থায় ITEM সংযোজন করলে $FRONT = REAR = 1$ হবে।

ii) Queue টি যদি Circular Queue হয় এবং $REAR = N$ হলে $REAR = 1$ হবে। অর্থাৎ, আবার প্রথম অবস্থানে ITEM সংযোজন হবে।

iii) যেকোনো অবস্থানে যদি ফাঁকা জায়গা থাকে, তবে সেই অবস্থানে ITEM সংযোজন হবে এবং REAR এর মান 1 বৃদ্ধি পাবে।

3. যেহেতু REAR হচ্ছে সর্বশেষ উপাদানের Address. তাই নতুন যে ITEM Insert হবে, তা Queue এর REAR এ অবস্থান করবে।

*** ২। Queue তে উপাদান বিয়োজনের Algorithm লেখ।

BCS- 36th, বাকাশিবো- ২০০০, ০৬, ০৮, ১৪'পর, ১৬, ১৬'পর, ১৮'পর, ১৯

উত্তর : Algorithm :

Step 1 : If FRONT = NULL then print Underflow and Return.

Step 2 : Set ITEM = QUEUE [FRONT]

Step 3 : if FRONT = REAR then

Set FRONT = REAR = NULL

else if FRONT = N then

Set FRONT = 1

else

Set FRONT = FRONT + 1

Step 4 : Return.

ব্যাখ্যা :

1. যেহেতু ITEM বিয়োজন করতে হবে, তাই প্রথমে শর্ত হচ্ছে Delete করার মত অবশ্যই Data বা ITEM থাকতে হবে। যদি FRONT = NULL হয়, তবে Queue খালি। তখন ITEM Delete করতে চাইলে Underflow হবে।

2. যে ITEM বিয়োজন করতে হবে তা Queue এর FRONT এ আছে।

3. i) যদি FRONT = REAR হয়, তবে Queue তে একটি উপাদান আছে। ঐ উপাদান Delete হলে FRONT = REAR = NULL হবে।

- ii) যদি Circular Queue হয় এবং $FRONT = N$ হয়, তবে Queue এর প্রথমদিকে খালি জায়গা আছে। তখন $FRONT = 1$ অর্থাৎ ১ম অবস্থান হতে $FRONT$ এর মান হবে।
- iii) যেকোনো অবস্থানে যদি ITEM থাকে, তবে তা Delete করলে $FRONT$ এর মান 1 বৃদ্ধি পাবে।

*** ৩। Stack এবং Queue এর মাঝে পার্থক্য লেখ।

BPSC-17, BRTA-12, DUET-05-06; বাকাশিবো- ২০১২, ১৫, ১৫' পরি, (১৭-২২)

উত্তর : Stack এবং Queue এর মাঝে পার্থক্য নিম্নরূপ :

Stack	Queue
i) LIFO পদ্ধতিতে কাজ করে।	i) FIFO পদ্ধতিতে কাজ করে।
ii) ডাটা সংযোজন ও বিয়োজন একই প্রান্তে সম্পন্ন হয়।	ii) ডাটা সংযোজন শেষে হয় এবং ডাটা বিয়োজন প্রথম থেকে হয়।
iii) TOP নামে একটি Variable ব্যবহৃত হয়।	iii) FRONT ও REAR নামে দুটি Variable ব্যবহৃত হয়।
iv) Data insert করলে TOP এর মান 1 করে বাড়ে এবং Data Delete করলে TOP এর মান 1 করে কমে।	iv) Data insert করলে REAR এর মান বাড়ে এবং Data Delete করলে FRONT এর মান বাড়ে।
v) Stack এ Data insert কে PUSH এবং Data Delete করাকে POP বলে।	v) Queue এ Data insert কে Enqueue এবং Data Delete করাকে Dequeue বলে।

vi) TOP = MAXSTK হলে Stack Full বা পরিপূর্ণ।	vi) FRONT = 1, REAR = N হলে Queue Full বা পরিপূর্ণ।
vii) TOP = 0 হলে Stack Empty বা খালি।	vii) FRONT = REAR = 0 হলে Queue Empty বা খালি।

*** 8। LIFO এবং FIFO এর মাঝে পার্থক্য লেখ।

বাকাশিৰো- ২০০৮'পরি, ১০, ১০'পরি, ১৭, ১৭'পরি, ২৩

উত্তর : LIFO এবং FIFO এর মাঝে পার্থক্য নিম্নরূপ :

LIFO	FIFO
i) LIFO এর পূর্ণরূপ Last In First Out.	i) FIFO এর পূর্ণরূপ First In First Out.
ii) Stack Data Structure এ LIFO পদ্ধতি ব্যবহৃত হয়।	ii) Queue Data Structure এ FIFO পদ্ধতি ব্যবহৃত হয়।
iii) Data Insert ও Delete একপ্রান্ত থেকেই হয়।	iii) Data Insert হয় শেষে এবং Data Delete হয় শুরুর প্রান্ত হতে।
iv) উদাহরণ : বইয়ের স্তুপ, প্লেটের স্তুপ ইত্যাদি।	iv) উদাহরণ : গাড়িতে উঠার লাইন, যেকোনো কাউন্টারের লাইন ইত্যাদি।

SOS**রচনামূলক প্রশ্নোত্তর :**

*** ১। Queue এ ডাটা সংযোজন এবং বিয়োজন পদ্ধতি চিত্রসহ বর্ণনা কর।

বাকশির্বো- ২০০৯, ০৯' পরি, ১৪' পরি

উত্তর : Queue এ ডাটা সংযোজন এবং বিয়োজন পদ্ধতি একটি উদাহরণের মাধ্যমে বর্ণনা করা হলো :

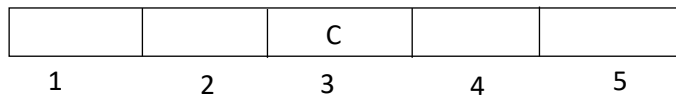
মনে করি, 5 টি উপাদানবিশিষ্ট Queue টি প্রাথমিক অবস্থায় খালি। এ অবস্থায় নিচের Operation গুলি করতে হবে।

i) Insert A, B, C



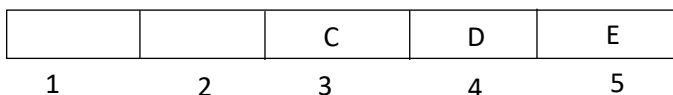
FRONT = 1
REAR = 3

ii) Delete B



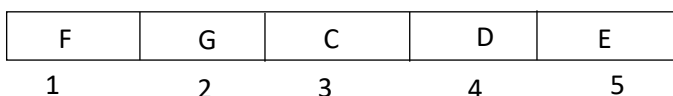
FRONT = 3
REAR = 3

iii) Insert D, E



FRONT = 3
REAR = 5

iv) Insert F, G



FRONT = 3
REAR = 2

v) Delete C

F	G		D	E
1	2	3	4	5

FRONT = 4
REAR = 2

vi) Insert H

F	G	H	D	E
1	2	3	4	5

FRONT = 4
REAR = 3

vii) Insert I : Overflow.

উল্লেখ্য FRONT = প্রথম উপাদানের Address ধারণ করে এবং Delete করলে FRONT এর মান বাড়ে।

এবং REAR = সর্বশেষ উপাদানের Address ধারণ করে এবং Insert করলে REAR এর মান বাড়ে।

অধ্যায়-৬

লিঙ্কড লিস্ট

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। **Linked List** বলতে কী বুঝায়?

বাকাশিবো- ২০০০, (০৭-০৯), ০৮'পরি, (১২-১৫)'পরি, ১৬, ২০, ২২, ২৩

উত্তর : Linked list হলো এক ধরনের Linear Data Structure, যেখানে Data গুলিকে একটার পরে আরেকটা লিংক আকারে লেখা হয়। Data রাখার জন্য একাধিক Field এর সমন্বয়ে Node তৈরি করা হয়। একটি Node হতে পরবর্তী Node-এ যাওয়ার জন্য লিংক থাকে।

** ২। লিঙ্কড লিস্টের নোডে কী কী অংশ থাকে?

বাকাশিবো- ২০০৭, ২১

উত্তর : লিঙ্কড লিস্টের নোডে দুটি অংশ থাকে। যথা :

i) Data বা Info

Data

NPA

node

ii) NPA = Next Pointer Address.

*** ৩। **Dynamic Memory Allocation** বলতে কী বুঝায়?

বাকাশিবো- ২০১০, ১০'পরি, ১২, ১৮

উত্তর : Program এর Run Time এ Memory Allocate করার প্রক্রিয়াকে Dynamic Memory Allocation বলে।

C Programming Language এ চারটি function ব্যবহার করে Dynamic Memory Allocate করা যায়। যথা :

i) malloc ()

ii) calloc ()

iii) free ()

iv) realloc ()

ব্যাখ্যা : আমরা যখন একটি Array ঘোষণা করি, তখন কোনো কোনো সময় অনেক বড় সাইজের Array ঘোষণা করি। যার বেশিরভাগই লাগে না। সেক্ষেত্রে মেমরির অপচয় হয়। আবার, অনেক সময় অনেক ছোট সাইজের Array ঘোষণা করি, যা পরবর্তীতে আর বর্ধিত করা যায় না। এ সমস্যাগুলি Dynamic Memory Allocation এর মাধ্যমে সমাধান করা যায়।

SOS**সংক্ষিপ্ত প্রশ্নোত্তর :**

***১। **Linked List** এর সীমাবদ্ধতা লেখ। বাকাশির্বো- ২০০৬, ০৯, ০৯'পরি

উত্তর : **Linked List** এর সীমাবদ্ধতা নিচে দেওয়া হলো :

- i) Pointer ব্যবহার করা হয় বলে অতিরিক্ত মেমরির প্রয়োজন।
- ii) Linked List এর কোনো উপাদান সরাসরি Access করা যায় না।
- iii) Linked List এর কোনো উপাদান এলোমেলো বা Random করা যায় না।
- iv) প্রতিটি Node তৈরিতে অনেক সময় প্রয়োজন হয়।

*** ২। **Linked List** এর সুবিধা ও অসুবিধাগুলো লেখ।

BPSC- 2013, DUET: 2015-16,

বাকাশির্বো- ২০০৬, ০৬'পরি, ০৭, ০৯, ০৯'পরি, ১১'পরি, ১৬'পরি, ১৮, ২০, ২১

উত্তর : **Linked List** এর সুবিধাসমূহ :

- i) মেমরির অপচয় হয় না।
- ii) যেকোনো সংখ্যক ডাটা নিয়ে কাজ করা যায়।
- iii) Data সংযোজন ও বিয়োজন প্রক্রিয়া সহজ।
- iv) Linear Data Structure.

যেমন : Stack, Queue ইত্যাদি Implement করা যায়।

- v) যেকোনো Type এর Data কে Node এ যুক্ত করা যায়।
- v) মেমরির সাইজ পূর্বেই নির্ধারণের প্রয়োজন হয় না।
- vii) Contiguous (পরপর) Memory Allocation এর প্রয়োজন হয় না।

Linked List এর অসুবিধাসমূহ :

- i) Random Access সম্ভব নয়।
- ii) Linear বা Single Linked List এ শুধুমাত্র Forward Traversing সম্ভব।
- iii) Binary Search এ ব্যবহার উপযোগী নয়।
- iv) অতিরিক্ত Pointer ব্যবহারের প্রয়োজন হয়।

*****৩। Linked List কে Memory তে উপস্থাপনের কৌশল চিত্রসহ বর্ণনা কর।**

বাকশির্বো- ২০০৪, ০৬, ০৯, ০৯'পর, ১১, ১১'পর, ১৪, ১৪'পর, ১৬, ২২

বিঃদ্রঃ এই প্রশ্নটি রচনামূলক প্রশ্ন হিসেবেও খুবই গুরুত্বপূর্ণ।

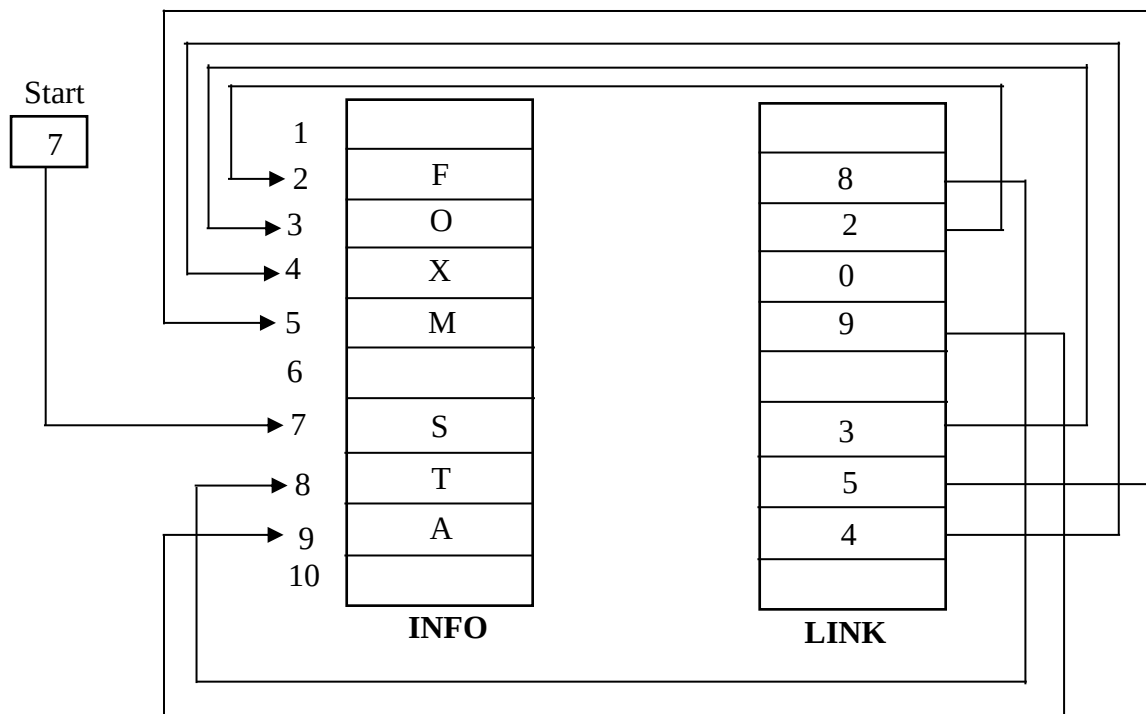
উত্তর : মনে করি, SOS একটি linked list, যা Memory তে উপস্থাপন করতে হবে।

আমরা জানি, Linked list এর Node এর দুটি অংশ।

- i) INFO = List এর Information বা Data ধারণ করে।
- ii) LINK = Next Pointer Address, পরবর্তী কোন Node এর সাথে সংযুক্ত তা নির্দেশ করে।

Linked list এর প্রথম উপাদানের লোকেশনকে নির্দেশ করার জন্য START নামক একটি Variable এবং List এর সমাপ্তি বুঝানোর জন্য Null ব্যবহার করা হয় Next Pointer Address এ।

যেমন : আমরা SOFTMAX কে Linked List এর মাধ্যমে Memory তে উপস্থাপন করতে চাচ্ছি।



এখানে,

START = 7, INFO [7] = S, যা Node এর প্রথম Character

LINK [3] = 3, INFO [3] = 0

LINK [2] = 2, INFO [2] = F

LINK [8] = 8, INFO [8] = T

LINK [8] = 5, INFO [5] = M

LINK [5] = 9, INFO [9] = A

LINK [9] = 4, INFO [4] = X

LINK [4] = 0 বা NULL

*** 8 | **Linked list** এবং **Array** এর মধ্যে পার্থক্য লেখ।

বাকাশিবো- ২০০৩, ০৬, ০৮, ১০, ১০'পরি, ১৫, ১৮, ১৯'পরি

BNPC- 2014, IPM- 2014, DUET: 2011-12

উত্তর : Linked list এবং Array এর মধ্যে পার্থক্য নিম্নরূপ :

Linked List	Array
i) Dynamic Memory Allocation পদ্ধতি ব্যবহৃত হয়।	i) Static Memory Allocation পদ্ধতি ব্যবহৃত হয়।
ii) ডাটার Size পূর্বেই নির্ধারণের প্রয়োজন হয় না।	ii) ডাটার Size পূর্বেই নির্ধারণের প্রয়োজন হয়।
iii) ডাটা সংযোজন ও বিয়োজন প্রক্রিয়া তুলনামূলকভাবে সহজ।	iii) ডাটা সংযোজন ও বিয়োজন প্রক্রিয়া তুলনামূলকভাবে জটিল।
iv) মেমরির অপচয় হয় না।	iv) মেমরির অপচয় হয়।
v) অতিরিক্ত Pointer Variable প্রয়োজন।	v) Pointer Variable প্রয়োজন নেই।
vi) যেকোনো Type এর ডাটা Node এ সংযুক্ত করা যায়।	vi) একই Type এর ডাটা নিয়ে কাজ করতে হয়।

** ৫ | **Linked list** এর **Traversing Algorithm** লেখ।

বাকাশিবো- ২০০০, ০২, ০৮, ০৯, ০৯'পরি, ১০, ১০'পরি, ১৩'পরি, (১৭-২০)

উত্তর : **Traversing Algorithm :**

Step 1 : Start

Step 2 : Set PTR = START

Step 3 : Repeat Steps 4 & 5 While PTR $\neq$ NULL

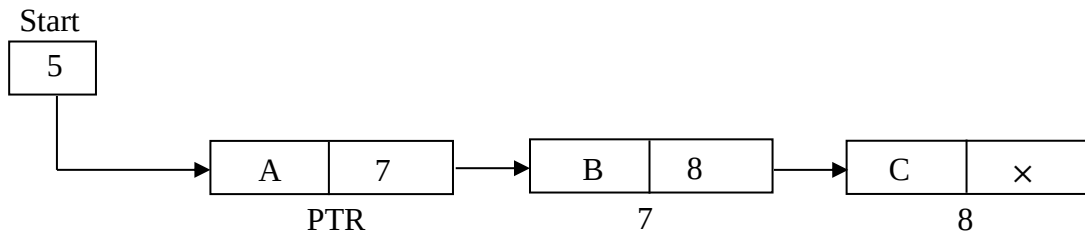
Step 4 : Apply process to INFO [PTR]

Step 5 : Set PTR = LINK [PTR]

[End of step 3 Loop]

Step 6 : End

ব্যাখ্যা :



মনে করি, তিন Node বিশিষ্ট উপরোক্ত linked list হতে সকল Data process করতে হবে।

1. PTR = START = 5, প্রথম Node.
2. যতক্ষণ পর্যন্ত PTR এর মান NULL না হয় ততক্ষণ পর্যন্ত loop ঘুরবে।
3. INFO [PTR] = A, LINK [5] = 7
 INFO [7] = B, LINK [7] = 8
 INFO [8] = C, LINK [8] = NULL
4. PTR এর মান NULL হয়ে গেছে। সুতরাং loop সমাপ্তি করতে হবে।

SOS রচনামূলক প্রশ্নোত্তর :

*** ১। Linked List এর Searching Algorithm লেখ।

বাকাশিবো- ২০১৮, ২০, ২২

উত্তর : Algorithm :

Step 1 : Set PTR = START

Step 2 : Repeat Step 3 while PTR $\neq$ NULL

Step 3 : If ITEM = INFO [PTR] then

Set LOC = PTR and Exit

else

Set PTR = LINK [PTR]

[End of if structure]

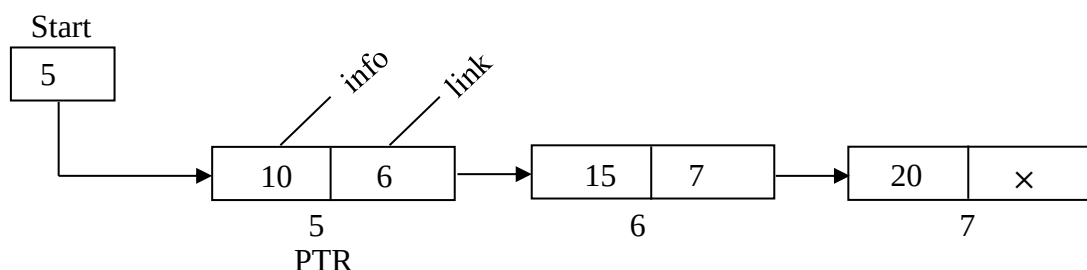
[End of Step 2 loop]

Step 4 : [Search is unsuccessful]

Set LOC = NULL

Step 5 : Exit

ব্যাখ্যা :



1. PTR = Start, Node এর Starting Address ধারণ করা।
2. প্রথম থেকে শুরু করে যে পর্যন্ত PTR এর মান NULL না হয়, অর্থাৎ সর্বশেষ Node পর্যন্ত ITEM (Data) কে Search করতে হবে।
3. যদি ITEM এবং ১ম Node এর Info বা Data সমান হয়, তবে সেই Data ১ম Node এ খুঁজে পাওয়া যাবে। সেই Location print করবে। যদি শর্ত মিথ্যা হয় তবে পরবর্তী Node এ গিয়ে পুনরায় চেক করবে। এভাবে যতক্ষণ পর্যন্ত NULL না পায় ততক্ষণ পর্যন্ত খুঁজতে থাকবে।
4. যদি LOC = NULL হয়, তবে Unsuccessful search অর্থাৎ আমরা যে Data খুঁজতেছি, তা linked list এ বিদ্যমান নেই।

*** ২। **Linked list** এ **Data** অথবা **Node** সংযোজন প্রক্রিয়া উদাহরণসহ বর্ণনা কর।

বাকশির্ষো- ২০০৫, ০৯, ০৯'পরি, ১০, ১৩'পরি, ১৪, ১৪'পরি, ১৭'পরি, ১৯

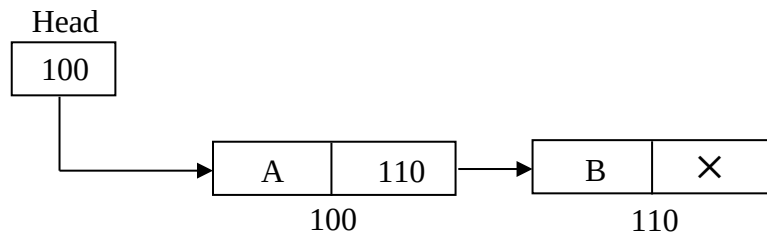
উত্তর : Linked list এ Node সংযোজনের জন্য তিনটি অবস্থানে Node সংযোজন করা যায়। যেমন :

- শুরুতে
- যেকোনো Node এর আগে বা পরে এবং
- শেষে

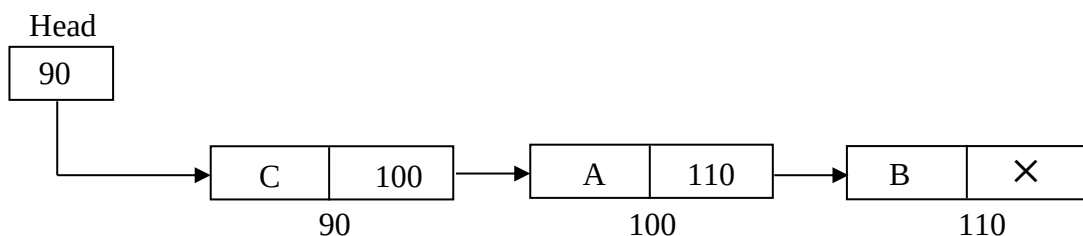
Linear linked list এর ক্ষেত্রে :

i) শুরুতে Node সংযোজন :

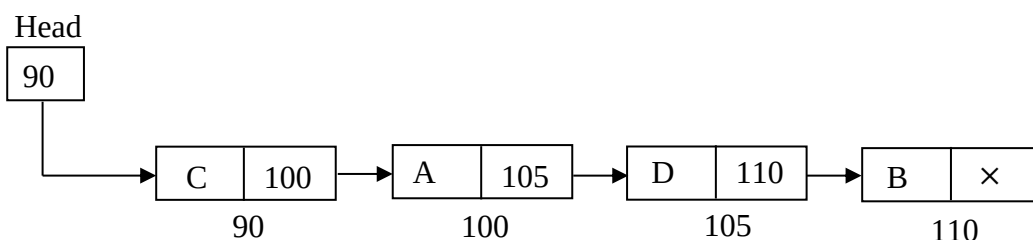
প্রাথমিক অবস্থায় :



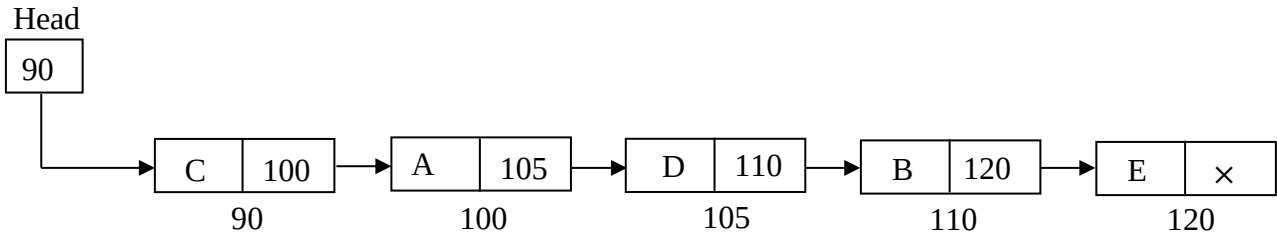
শুরুতে Node সংযোজনের পরের অবস্থা :



ii) যেকোনো Node এর আগে বা পরে Node সংযোজন :



iii) শেষে Node সংযোজন :



***৩। Linked List এর শুরুতে Node সংযোজনের Algorithm

লেখ।

বাকশিবো- ২০০০, ০৭, ০৮, ০৮'পরি, ১২, ১৬, ১৯'পরি, ২৩

উত্তর : Algorithm :

Step 1 : if PTR = NULL then

Print Overflow & Return

else

PTR = (node *) malloc (size of (node))

Step 2 : Set PTR → info = ITEM

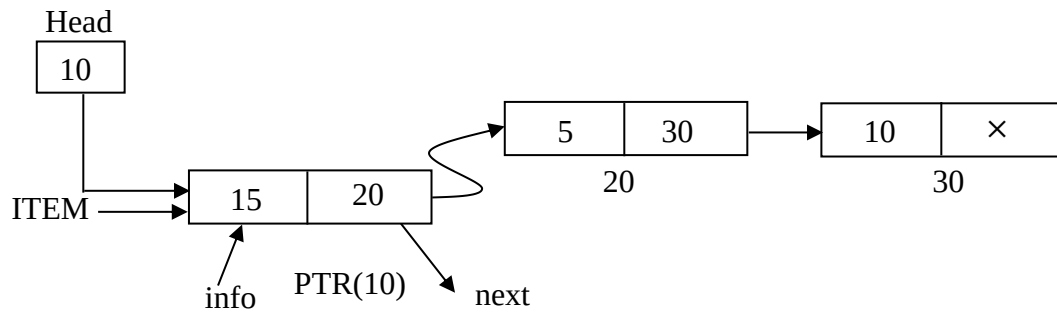
Step 3 : Set PTR → next = Head

Step 4 : Set Head = PTR

ব্যাখ্যা :

1. যে Node Insert করতে হবে, তাকে PTR নাম দেওয়া হলো। এখন PTR এর মান যদি NULL হয় অর্থাৎ Insert করার মত যদি কোনো Node না থাকে তবে Overflow print হবে। যদি PTR এর মান NULL না হয়, তবে Dynamic Memory Allocation এর মাধ্যমে Node সংযোজন হবে।
2. PTR এর Info তে যে Data Insert হবে, তাকে ITEM বলব।
3. PTR এর Next Pointer Address কে Head Assign হলো।
4. Head কে PTR করা হলো।

উদাহরণ :



**** 8 । Linked List হতে নোড বিয়োজন প্রক্রিয়া বর্ণনা কর ।**

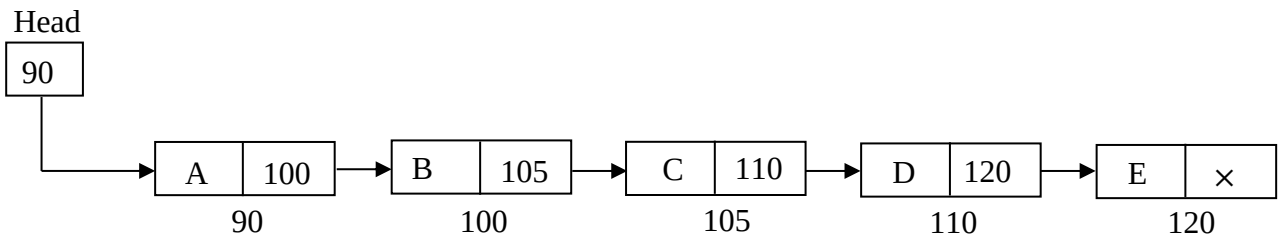
বাকাশিবো- ২০০৭, ১১, ১৮'পরি

উত্তর : Linked List হতে নোড বিয়োজন তিনটি অবস্থান থেকে করা যায় ।

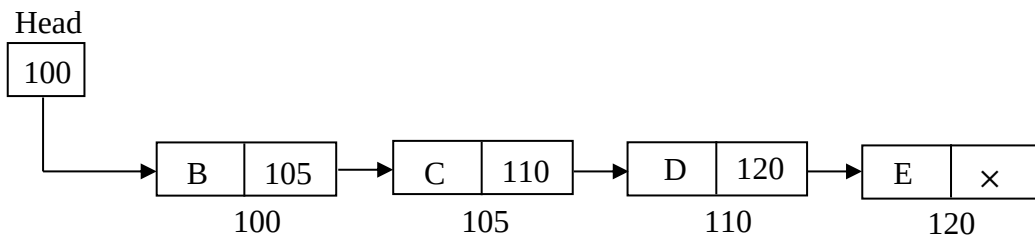
i) শুরুতে ii) যেকোনো Node এর পূর্বে বা পরে এবং iii) শেষে

Linear linked List এর ক্ষেত্রে :

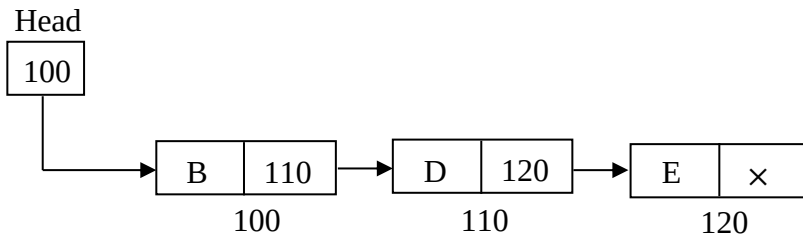
প্রাথমিক অবস্থায় :



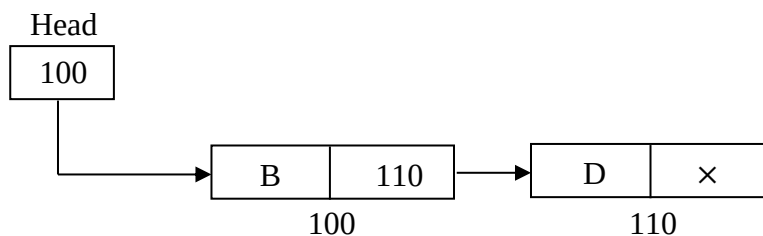
i) শুরুতে Node বিয়োজন :



ii) যেকোনো **Node** এর পূর্বে বা পরে **Node** বিয়োজন :



iii) শেষে **Node** বিয়োজন :



*** ৫। **Linked List** হতে নোড বিয়োজনের **Algorithm** লেখ।

বাকাশিৰো- ২০০৮, ০৮'পরি, ০৯, ০৯'পরি, ১১'পরি, ১৩'পরি, ১৭'পরি

উত্তর : Linked List এর শুরুতে Node বিয়োজনের Algorithm :

Step 1 : if Head = NULL then

Print Overflow & Return

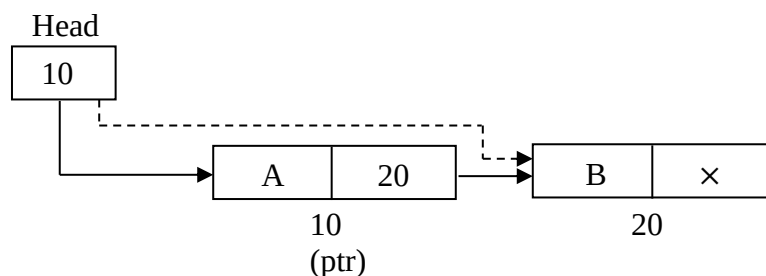
Step 2 : Set PTR = Head

Step 3 : Set Head = Head next

Step 4 : Print element delete is PTR → info

Step 5 : free (ptr)

উদাহরণ :



অধ্যায়-৭

দ্বি-ডাটা স্ট্রাকচার

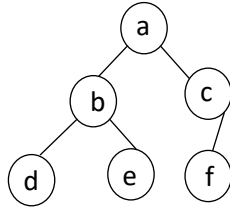
SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। **Tree Data Structure** বলতে কী বুঝায়? বাকাশিবো- ২০১৮, ১৯

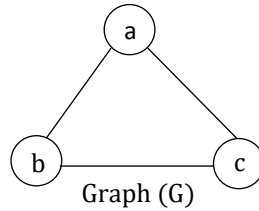
উত্তর : Tree এক ধরনের Non-Linear Data Structure, যেখানে Data গুলি Hierarchical ভাবে Node আকারে সংযুক্ত থাকে এবং কোনো সাইকেল উৎপন্ন করে না।

যেমন :

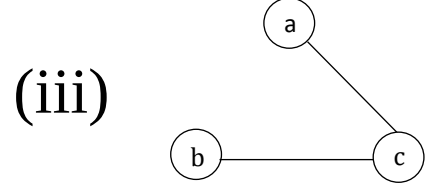
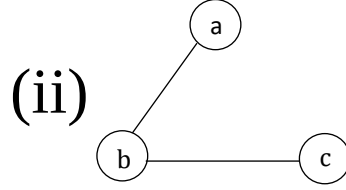
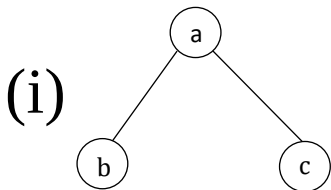
** ২। **Spanning Tree** কাকে বলে? BCS- 36th, বাকাশিবো- ২০১৯, ২৩

উত্তর : একটি গ্রাফ এর সাব-সেটকে Spanning Tree বলে।

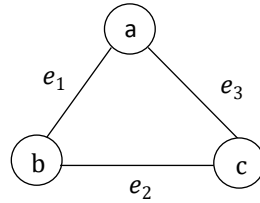
যেমন :



উপরের Graph এর তিনটি Spanning Tree তৈরি করা যায়।



Note : Graph : Graph এক ধরনের Non-Linear Data Structure যেখানে কিছু Vertex থাকে এবং কিছু Edge থাকে এবং Vertex গুলো Edge দ্বারা Connected থাকে।



চিত্রে, Vertex = a, b, c এবং Edge = e_1, e_2, e_3

*** ৩। Binary Tree কী?

DUET- 15-16

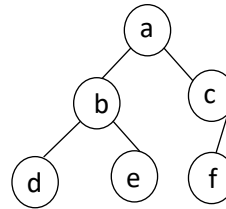
উত্তর : যে Tree এর Child এর সংখ্যা সর্বোচ্চ দুটি, তাকে Binary Tree বলে।

যেমন :

ব্যাখ্যা : a এর Child দুইটি (b, c)

b এর Child দুইটি (d, e)

c এর Child একটি (f)



Binary Tree এর ক্ষেত্রে Child এর সংখ্যা সর্বোচ্চ দুইটি। অর্থাৎ দুই এর চেয়ে কমও হতে পারে। কিন্তু দুই এর চেয়ে বেশি হবে না।

SOS**সংক্ষিপ্ত প্রশ্নোত্তর :**

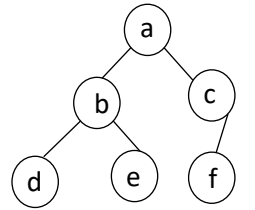
*** ১। Tree এর প্রকারভেদ বর্ণনা কর। অথবা, বিভিন্ন প্রকার tree-
এর data structure চিত্র সহ বর্ণনা কর।

বাকাশিবো- ২০২৩

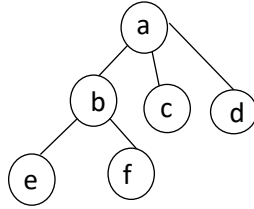
উত্তর : Tree কে বিভিন্ন ভাগে ভাগ করা যায়। যেমন :

i) M-Array Tree : যে Tree এর M বা তার কম সংখ্যক Child থাকে, তাকে M-Array Tree বলে। এখানে M এর মান 2, 3.....ইত্যাদি হতে পারে।

যেমন : $M = 2$ হলে Child এর সংখ্যা সর্বোচ্চ দুইটি হবে।

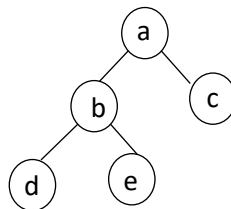


$M = 3$ হলে Child এর সংখ্যা সর্বোচ্চ তিনটি হবে।



(ii) Full M-Array Tree : যে M-Array Tree তে প্রতিটি Node এ Child এর সংখ্যা M সংখ্যক তাকে Full M-Array Tree বলে।

যেমন :



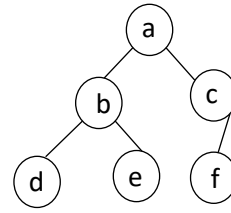
(iii) Binary Tree : যে Tree এর Child এর সংখ্যা সর্বোচ্চ দুটি, তাকে Binary Tree বলে।

যেমন :

ব্যাখ্যা : a এর Child দুইটি (b, c)

b এর Child দুইটি (d, e)

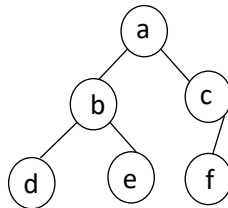
c এর Child একটি (f)



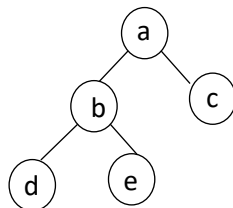
Binary Tree এর ক্ষেত্রে Child এর সংখ্যা সর্বোচ্চ দুইটি। অর্থাৎ দুই এর চেয়ে কমও হতে পারে। দুই এর চেয়ে বেশি হবে না।

(iv) Complete Binary Tree : Complete Binary Tree এমন এক ধরনের Binary Tree, যেখানে সর্বশেষ Level গুলি বাম হতে ডানে পরিপূর্ণ করতে হবে।

যেমন :

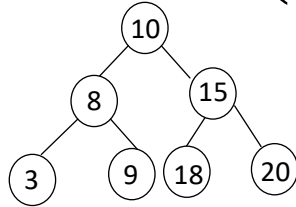


(v) Extended Binary Tree : যে Binary Tree তে 0 অথবা 2 টির কম বা বেশি Child থাকবে না, তাকে Extended Binary Tree বলে। যেমন :



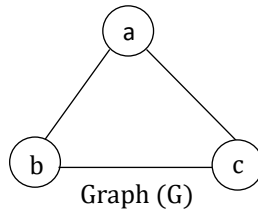
(vi) Binary Search Tree : যে Binary Tree এর Root Node এর মান Left Child থেকে বড় এবং Right Child থেকে ছোট, তাকে Binary Search Tree (BST) বলে।

যেমন :

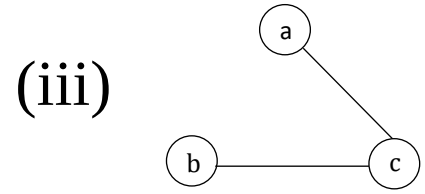
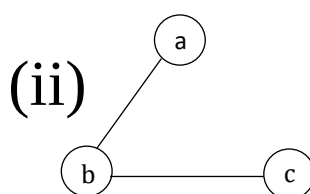
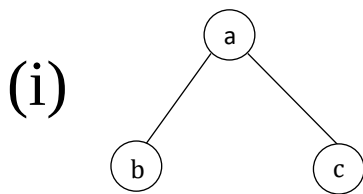


(vii) Spanning Tree : একটি গ্রাফ এর সাব-সেটকে Spanning Tree বলে।

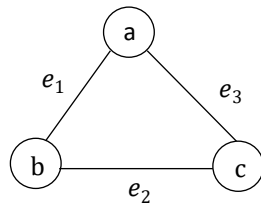
যেমন :



উপরের Graph এর তিনটি Spanning Tree তৈরি করা যায়।



Note : Graph : Graph এক ধরনের Non-Linear Data Structure যেখানে কিছু Vertex থাকে এবং কিছু Edge থাকে এবং Vertex গুলো Edge দ্বারা Connected থাকে।

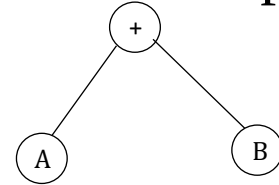


চিত্রে, Vertex = a, b, c এবং Edge = e_1, e_2, e_3

Expression Tree

যে Tree এর Node গুলি Operator এবং Leaf গুলি Operand, তাকে Expression Tree বলে।

যেমন : $A + B$ এর Expression Tree



এখানে, $+$ = Operator, যা Tree এর Node বা Root. এবং

A, B = Operand, যা Tree এর Leaf অর্থাৎ এরপর Tree এর আর কিছু নেই।

Expression Tree আঁকার সময় একাধিক Operator একসাথে থাকলে Operator Precedence বা অপারেটর অগ্রগণ্যতা অনুসারে করতে হবে।

যেমন :

Operator	Operator precedence
সূচক ($\uparrow$)	1
Parenthesis ()	2
$*, /$	3
$+, -$	4

**** ২। Expression Tree আকারে প্রকাশ কর।**

বাকাশিবো- ২০১৯, ২২

উত্তর : i) $2x + 7$

ii) $(2a + b)(4a - 5b)$

iii) $(3x + 4z)^3 + (2x - y)$

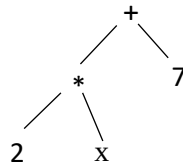
iv) $(4x - 9b)^3 + (2A - 7B)^2$

v) $(ab + xy)(2x + ac)^3$

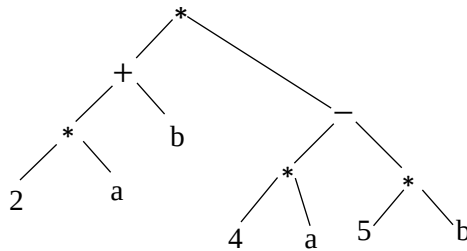
vi) $(x + y + z^3)^4 (xy + yz + zx)$

vii) $(4x - 9b)^2 (A - 7B) + 2C$

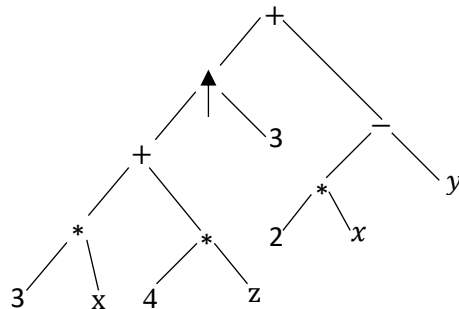
(i) $2x + 7$ এর Expression Tree :



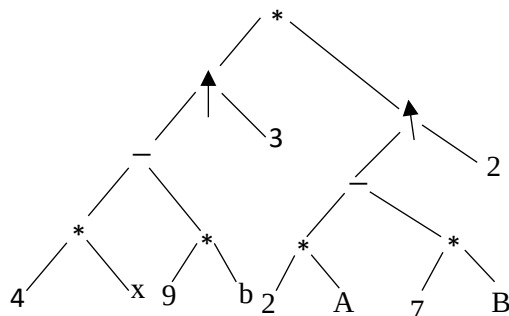
(ii) $(2a + b)(4a - 5b)$ এর Expression Tree :



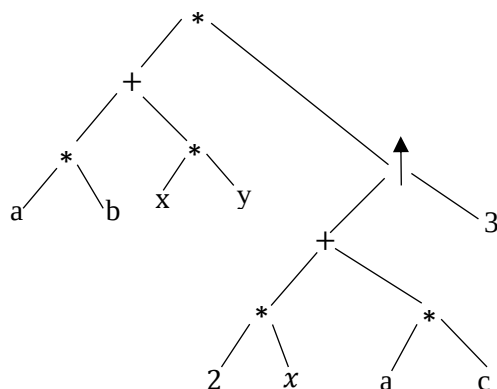
(iii) $(3x + 4z)^3 + (2x - y)$ এর Expression Tree :



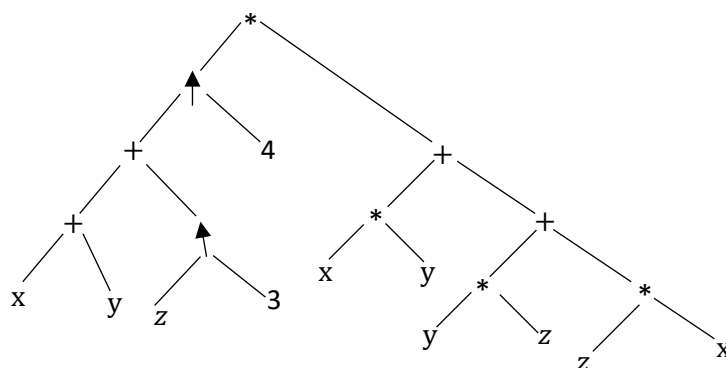
(iv) $(4x - 9b)^3 + (2A - 7B)^2$ এর Expression Tree :



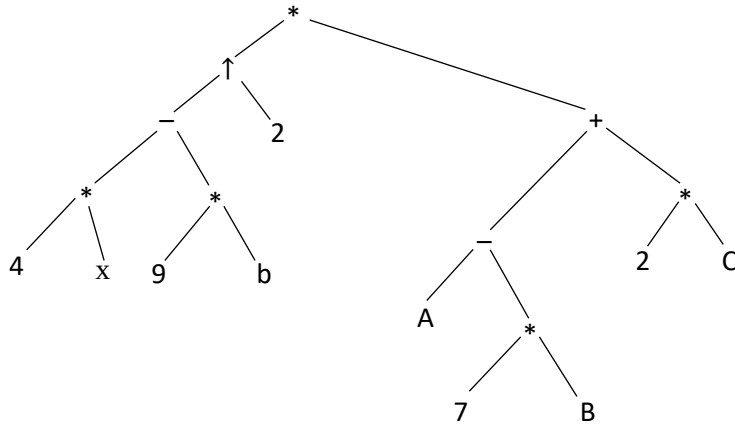
(v) $(ab + xy)(2x + ac)^3$ এর Expression Tree :



(vi) $(x + y + z^3)^4 (xy + yz + zx)$ এর Expression Tree :



(vii) $(4x-9b)^2 (A-7B) + 2C$ এর Expression Tree :



*** ৩। **Tree Traversal** বলতে কী বুঝে? উদাহরণসহ বর্ণনা কর।

তথ্য যোগাযোগ ও প্রযুক্তি বিভাগ- ২০১৯

উত্তর : **Tree Traversal :** যে সকল পদ্ধতিতে কোনো একটি Tree এর সকল Node Traverse বা Visit করা হয়, তাকে Tree Traversal বলে। Tree Traversal তিন ভাবে করা যায়। যথা :

i) In-order : এ পদ্ধতিতে left Sub-tree সবার আগে Visit করতে হয়, তারপর Root এবং সর্বশেষে right Sub tree Visit করতে হয়।

ii) Pre-order : এ পদ্ধতিতে Root সবার আগে Visit করতে হয়। তারপর Left sub-tree এবং সর্বশেষে Right Sub-tree Visit করতে হয়।

iii) Post-order : এ পদ্ধতিতে সবার আগে Left sub-tree, পরে Right Sub-tree Visit করতে হয় এবং সর্বশেষে Root Visit করতে হয়।

Note : সহজে মনে রাখার উপায় :

(i) In-order : In মানে মাঝে । Root এর অবস্থান মাঝে ।

Left $\longrightarrow$ Root $\longrightarrow$ Right

(ii) Pre-order : Pre মানে পূর্বে। Root এর অবস্থান প্রথমে।

Root $\longrightarrow$ Left $\longrightarrow$ Right

(iii) Post-order : Post মানে পরে । Root এর অবস্থান শেষে ।

Left $\longrightarrow$ Right $\longrightarrow$ Root

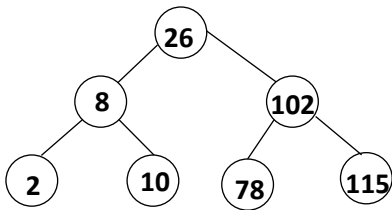
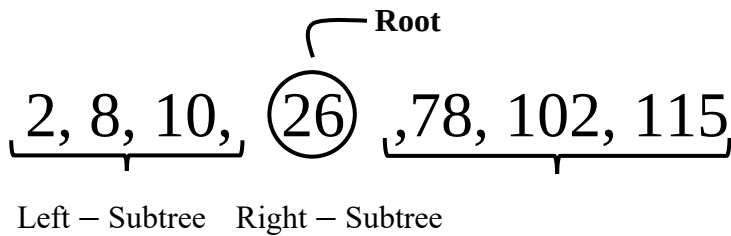
**** 8 | Binary Search Tree (BST) তৈরি কর।**

BNPC- 2014, BCS- 36th, DUET: 2005-06, 09-10

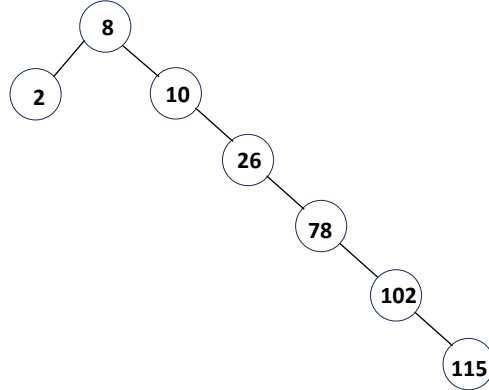
উত্তর : (i) 8, 10, 26, 2, 78, 102, 115

(ii) 7, 9, 25, 1, 77, 100, 112

(i) প্রথমে মানগুলিকে Ascending order বা ছোট থেকে বড় অনুসারে সাজাতে হবে।

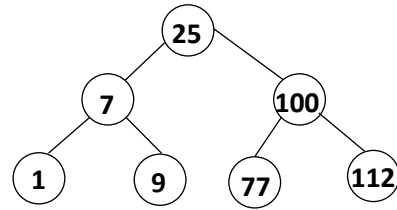
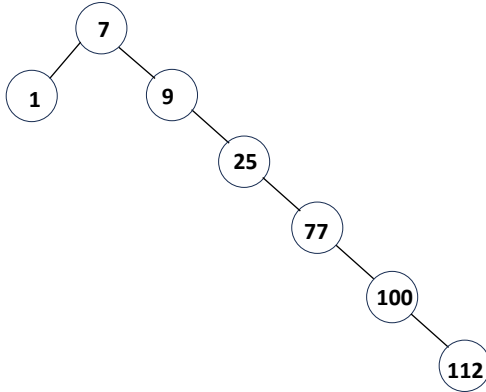


অথবা, নিচের নিয়ম অনুযায়ীও BST তৈরি করা যায়।



(ii) 1, 7, 9, (25), 77, 100, 112

অথবা,

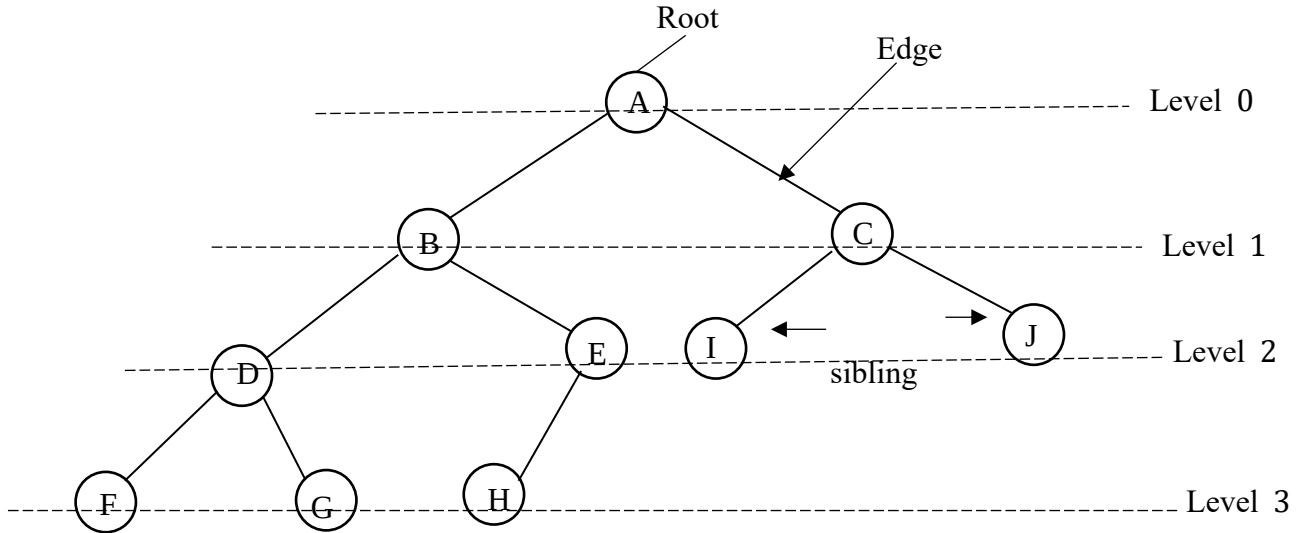


Note : Ascending order এ সাজানোর পর মাঝের মানকে Root ধরতে হবে। উপাদান সংখ্যা যদি জোড় হয় তবে মাঝের 2টা হতে যেকোনো একটিকে Root ধরতে হবে। Root এর বামপাশের সকল উপাদানকে Left Sub-tree এবং ডানপাশের সকল উপাদানকে Right Sub-tree ধরে BST তৈরি করতে হবে।

SOS**রচনামূলক প্রশ্নোত্তর :**

*** ১। Tree এর টারমিনোলজিগুলো বর্ণনা কর।

উত্তর : Tree এর টারমিনোলজিগুলো নিম্নে বর্ণনা করা হলো :



যে সকল Element বা উপাদানগুলি নিয়ে একটি Tree গঠিত, তাকে Tree এর Terminology বলে। নিচে তাদের বর্ণনা দেওয়া হলো :

i) Root : একটি Tree এর সবার উপরের Node কে Root বলে।

বাকাশিবো- ২০২৩

চিত্রে A = Root.

ii) Edge : যেকোনো দুইটি Node এর মাঝে সংযুক্ত পথকে Edge বলে।

iii) Node : Tree প্রতিটি উপাদানকেই Node বলে।

চিত্রে A, B, C, D, E, F, G, H, I, J সবই Node.

vi) Parent Node : কোনো Node এর যদি এক বা একাধিক Child থাকে, তাকে Parent Node বলে। চিত্রে, A, B, C, D, E Parent node.

v) Child Node : Root ছাড়া Tree এর বাকী সবই Node ই Child Node. চিত্রে B, C, D, E, F, G, H, I, J Child Node.

vi) Left child & Right child : একটি Tree এর বামপাশের Node গুলিকে Left child এবং ডানপাশের Node গুলিকে Right child বলে। চিত্রে B, D, E, F, G, H left child এবং C, I, J Right child

vii) Internal node/Vertex : যে সকল Vertex/node এর Child আছে, তাদেরকে Internal node/vertex বলে।

চিত্রে B, C, D, E হলো Internal vertex

viii) Leaf : যে সকল Node গুলির কোনো Child থাকে না, তাদেরকে Leaf বলে।

বাকাশিবো- ২০২৩

চিত্রে F, G, H, I, J হলো Leaf.

ix) Sibling : যে সকল Node গুলির Parent node একই তাদেরকে Sibling বা সহোদর বলে।

চিত্রে, (I, J Sibling), (D, E Sibling), (F, G Sibling)

x) Ancestor : কোনো একটি নির্দিষ্ট Node এর Root পর্যন্ত উপরের সকল Node কে Ancestor বা পূর্বসূরী বলে।

চিত্রে, F এর Ancestor D, B, A

J এর Ancestor C, A

xi) Descendant : একটি Node এর অধীনস্থ সকল Node কে ঐ Node এর Descendant বা উত্তরাধিকারী বলে।

চিত্রে B এর Descendant D, E, F, G, H

xii) Branch : যে সকল Node এর অন্তত (Minimum) একটি Child থাকে, তাকে Branch বলে।

চিত্রে A, B, C, D, E হলো Branch.

xiii) Degree : কোনো একটি Node এর Sub tree সংখ্যাকে ঐ Node এর Degree বলে।

চিত্রে A এর Sub tree 2টি।

xiv) Height of Node : কোনো একটি Node হতে একটি Leaf এ যেতে সর্বাধিক যে দূরত্ব অতিক্রম করতে হয়, তাকে Height of

Node বলে। চিত্রে B এর Height = 2, A এর height = 3

xv) Height of Tree : Root হতে leaf তে আসতে সর্বাধিক যে দূরত্ব অতিক্রম করতে হয়, তাকে Height of Tree বলে।

চিত্রে, Tree এর Height = 4

xvi) Depth : Root হতে কোনো একটি Node এ পৌঁছানোর জন্য Edge সংখ্যা।

চিত্রে, D এর Depth = 2

xvii) Level : Root হতে ঐ node এ পৌঁছানোর জন্য Edge সংখ্যায় চেয়ে 1 বেশি।

$\therefore \text{level} = \text{Depth} + 1$ (D এর level = 2)

xviii) sub tree: Tree এর সকল উপসেটকে sub tree বলে।

বাকশিবো-২০২৩

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

**** ১। সার্চিং (Searching) বলতে কী বুঝায়?** বাকাশিবো- ২০১৮, ১৮'পরি**উত্তর :** সার্চিং শব্দের আভিধানিক অর্থ খুঁজা। কোনো Data Structure হতে নির্দিষ্ট কোনো উপাদানকে খুঁজে বের করাকে সার্চিং বলে।**** ২। Binary Search কী?**

বাকাশিবো- ২০১৩'পরি, ২২

উত্তর : Binary Search এর ক্ষেত্রে সম্পূর্ণ ডাটাগুলোকে দুইভাগে ভাগ করে মধ্যবর্তী ডাটার সাথে তুলনা করে কাক্ষিত ডাটা খুঁজে বের করা হয়।**** ৩। Binary Search এর সুবিধা কী?**

বাকাশিবো- ২০১৭, ১৯'পরি

উত্তর : Binary Search যেহেতু ডাটাগুলিকে অর্ধেক করে দুই ভাগে ভাগ করে, সেহেতু Searching এ সময় অপেক্ষাকৃত কম লাগে।***** ৪। Binary Search এর সীমাবদ্ধতা (limitation) লেখ।**

বাকাশিবো- ২০০৯, ১১, ১৩'পরি, ১৪'পরি, ১৬'পরি, ২১

উত্তর : i) Unsorted Data এর ক্ষেত্রে প্রয়োগ করা যায় না।

ii) লিস্ট এর দৈর্ঘ্য অনেক বড় হয়।

****৫। Binary Search এর Complexity কত?** বাকাশিবো- ২০০৯, ২২**উত্তর :** $f(n) = \log_2 n$, $n =$ উপাদান সংখ্যা।

SOS

সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। **Linear Search Algorithm** লেখ।

বাকাশিবো- ২০১১, ১৩'পর, ১৪'পর

উত্তর : **Algorithm :**

LINEAR (DATA, N, ITEM, LOC)

Step 1 : Set DATA [n + 1] = ITEM

Step 2 : Set LOC = 1

Step 3 : Repeat while DATA [LOC] $\neq$ ITEM
then,

Set LOC = LOC + 1

Step 4 : if LOC = N + 1, then set LOC = 0

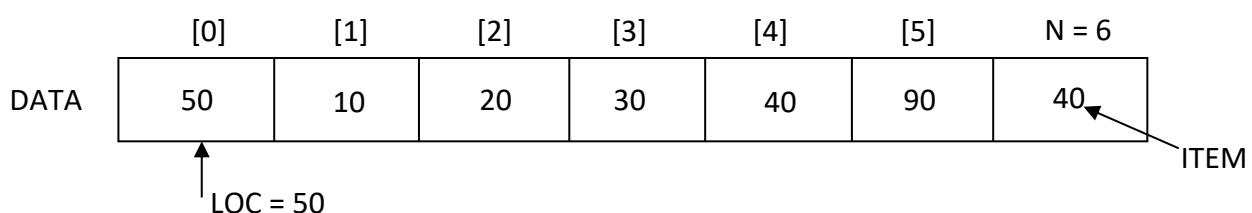
Step 5 : Exit

ব্যাখ্যা :এখানে, DATA = Array Name (ধরে নিলাম, যে Data গুলি
হতে আমরা Search করব, তা Array তে আছে)

N = Total Data Elements

ITEM = যে Data আমরা Search করব

LOC = Location (Search কৃত ডাটার Memory Location)



1. আমরা যে Data Search করব, তাকে $n + 1$ তম location এ insert করব।
2. LOC এর মান প্রথম অবস্থানে রাখতে হবে।
3. DATA [LOC] = ITEM না হওয়া পর্যন্ত Loop চলবে।

DATA [0] = 50,	ITEM = 40,	
50 $\neq$ 40,	LOC = LOC + 1	= 0 + 1 = 1
DATA [1] = 10,	10 $\neq$ 40,	LOC = 1 + 1 = 2
DATA [2] = 20,	20 $\neq$ 40,	LOC = 2 + 1 = 3
DATA [3] = 30,	20 $\neq$ 40,	LOC = 3 + 1 = 4
DATA [4] = 40,	40 = 40,	Successful Search

4. যদি LOC এর মান $n + 1$ হয়, তবে Unsuccessful Search.

*** ২। লিনিয়ার সার্চের **Average Case Complexity** নির্ণয় কর।

বাকাশিবো- ২০০২, ০৪, ০৮, ১০, ২১

অথবা, দেখাও যে, **Average Case** এর জন্য **Linear Search Algorithm** এর Complexity $f(n) = \frac{n+1}{2}$

বাকাশিবো- ২০০৪, ০৮, ১০, ১১, ১৫

উত্তর : Linear Search এর ক্ষেত্রে Average Case Complexity এর মান Probability বা সম্ভাব্যতার উপর নির্ভর করে।

মনে করি, ঘটনা ঘটার সম্ভাবনা = p

ঘটনা না ঘটার সম্ভাবনা = q

Probability হতে আমরা জানি,

কোনো ঘটনা ঘটার এবং না ঘটার সম্ভাবনার যোগফলের মান 1.

অর্থাৎ, $p + q = 1$

বা, $p_1 + p_2 + p_3 + \dots + p_n + q = 1$

$\therefore$ প্রথম ডাটা পেতে তুলনা করতে হয় $= \frac{1}{n}$, $n =$ মোট ডাটা

২য় ডাটা পেতে তুলনা করতে হয় $= \frac{2}{n}$

৩য় ডাটা পেতে তুলনা করতে হয় $= \frac{3}{n}$

n তম ডাটা পেতে তুলনা করতে হয় $= \frac{n}{n}$

সুতরাং, গড় তুলনার মান, $f(n) = 1.p_1 + 2.p_2 + 3.p_3 + \dots + np_n + (n+1)q$

$= \frac{1}{n} + \frac{2}{n} + \frac{3}{n} + \dots + \frac{n}{n} + 0$ [($n+1$) তম অবস্থানে Data নেই]

$= \frac{1}{n} (1 + 2 + 3 + \dots + n)$

$= \frac{1}{n} \left\{ \frac{n(n+1)}{2} \right\}$ $[1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}]$

$\therefore f(n) = \frac{n+1}{2}$ (Showed)

Note : Probability : Probability অর্থ সম্ভাব্যতা। কোনো একটি ঘটনা ঘটার সম্ভাবনা এবং ঘটনার মোট ফলাফলের অনুপাতকে Probability বলে।

$$\text{অর্থাৎ, Probability} = \frac{\text{ঘটনার অনুকূলে ফলাফল}}{\text{ঘটনার মোট ফলাফল}}$$

যেমন : একটি মুদ্রা নিক্ষেপ করলে Head আসার Probability = $\frac{1}{2}$

একটি ছক্কা নিক্ষেপ করলে জোড় সংখ্যা আসার Probability = $\frac{3}{6}$

একটি ছক্কা নিক্ষেপ করলে বিজোড় সংখ্যা আসার Probability = $\frac{3}{6}$
ইত্যাদি।

*** ৩। **Linear Search** এবং **Binary Search** এর পার্থক্য লেখ।

বাকশিবো- ২০০০, (০৭-১২), ০৭'পরি, ০৯'পরি, ১৪'পরি, ১৬, ১৬'পরি, ১৭, ২০

উত্তর : Linear Search এবং Binary Search এর মাঝে পার্থক্য নিম্নরূপ :

Linear Search	Binary Search
i) Sorted ও Unsorted দুই ক্ষেত্রেই ITEM Search করা সম্ভব।	i) শুধুমাত্র Sorted ডাটার ক্ষেত্রে ITEM Search করা যায়।
ii) একদিক থেকে পর্যায়ক্রমে Data তুলনা শুরু হয়।	ii) Data গুলোকে দুই ভাগে ভাগ করে Data তুলনা শুরু হয়।

iii) বেশি সংখ্যক তুলনার প্রয়োজন।	iii) তুলনামূলক কম সংখ্যকবার তুলনার প্রয়োজন।
iv) Searching এ সময় বেশি লাগে।	iv) Searching এ সময় কম লাগে।
v) Worst Case Complexity, $f(n) = n + 1$	v) Worst Case Complexity, $f(n) = \log_2 n + 1$
vi) Average Case Complexity, $f(n) = \frac{n+1}{2}$	vi) Binary search এর ক্ষেত্রেও প্রায় একই মানের।
vii) Algorithm Run হতে সর্বাধিক n এর সমানুপাতিক সময় প্রয়োজন।	vii) Algorithm Run হতে সর্বাধিক $\log_2 n$ এর সমানুপাতিক সময় প্রয়োজন।

*** 8। Binary Search এর সীমাবদ্ধতা লেখ।

DUET: 2017-18, বাকশি-২০১১, ১৫'পরি, ১৮, ২২

উত্তর : Binary Search এর সীমাবদ্ধতা নিম্নরূপ :

- শুধুমাত্র Sorted ডাটার ক্ষেত্রেই ITEM Search করা যায়।
- ক্রমানুসারে সাজানো থাকে বলে লিস্ট এর Length বড় হয়।
- নতুন Data সংযোজনে সময় বেশি লাগে।
- প্রচুর মেমরির প্রয়োজন।
- বেশি সংখ্যক ডাটার জন্য উপযোগী নয়।

**** ৫। Binary Search Algorithm এর Complexity নির্ণয় কর।**

বাকাশিবো- ২০০৯, ০৯'পরি, ১৪'পরি, ১৭'পরি

উত্তর : Binary Search Algorithm এর Complexity এর ক্ষেত্রে, n উপাদানবিশিষ্ট Array হতে ITEM কে Search করতে তুলনার

সংখ্যাকে $f(n)$ দ্বারা প্রকাশ করা হয়।

আমরা জানি, Binary Search এর ক্ষেত্রে, প্রতিবারে তুলনার জন্য মোট উপাদান গুলিকে দুই ভাগে ভাগ করা হয়। ফলে ITEM শনাক্ত করতে $f(n)$ সংখ্যক বার তুলনার প্রয়োজন।

$$\therefore f(n) = \log_2 n + 1$$

এক্ষেত্রে দেখা যাচ্ছে, Binary Search Algorithm এর Running Time প্রায় $\log_2 n$ এর সমানুপাতিক এবং এই Algorithm এর Worst Case, Average Case এবং Best Case এর ক্ষেত্রেও একই সময়ের প্রয়োজন।

SOS

রচনামূলক প্রশ্নোত্তর :

***** ১। Linear Searching পদ্ধতি বর্ণনাসহ Algorithm লেখ।**

খাদ্য মন্ত্রণালয়- ২০২২, বাকাশিবো- ২০০৬, ০৯'পরি, ১২, ১৯, ১৯'পরি, ২০

উত্তর : **Algorithm :**

LINEAR (DATA, N, ITEM, LOC)

Step 1 : Set DATA [$n + 1$] = ITEM

Step 2 : Set LOC = 1

Step 3 : Repeat while DATA [LOC] $\neq$ ITEM

then,

Set LOC = LOC + 1

Step 4 : if LOC = N + 1, then set LOC = 0

Step 5 : Exit

Linear Search পদ্ধতি : ধরা যাক, DATA একটি 'n' সংখ্যক Element (উপাদান) বিশিষ্ট Linear Array এবং DATA অ্যারে থেকে ITEM-কে লিনিয়ার সার্চ পদ্ধতিতে খুঁজে বের করতে হবে। ITEM হলো ঐ ডাটা, যাকে Search করতে হবে। ITEM-কে DATA অ্যারের প্রতিটি উপাদানের সাথে তুলনা করে দেখতে হবে যে, DATA অ্যারের কোন উপাদান ITEM-এর সমান হয়। যদি DATA অ্যারের কোনো উপাদান ITEM-এর সমান হয়, তবে সার্চকে সফল বলা হয় এবং কোন স্থানের উপাদান ITEM-এর সমান তা শনাক্ত করা হয়। যদি DATA অ্যারের কোনো উপাদান ITEM-এর সমান না হয়, তবে Search-কে Unsuccessful বলা হয়।

লিনিয়ার সার্চ সম্পাদন করার জন্য প্রথমত DATA [1] = ITEM কি না তা পরীক্ষা করা হয়। যদি DATA [1] = ITEM না হয়, তবে এরপর DATA [2] = ITEM কি না তা পরীক্ষা করা হয়। যদি DATA [2] = ITEM না হয়, তবে পরবর্তী উপাদানের সাথে একই পদ্ধতিতে পরীক্ষা সম্পন্ন করা হয়। এ পদ্ধতিতে Item-কে শনাক্ত করার জন্য DATA ধারাবাহিকভাবে সামনের দিকে চলতে থাকে। সেজন্যই এ পদ্ধতিকে লিনিয়ার সার্চ বলা হয়।

*** ২। **Binary Search** পদ্ধতির বর্ণনাসহ **Algorithm** লেখ।

বাকশিৰো- ২০০৪, ০৭,০৯, ০৯'পরি,১০, ১৪'পরি, ১৫,১৬, (১৬-১৯)'পরি, ২২

উত্তর : **Algorithm :**

Step 1 : Set $BEG = LB$, $END = UB$ and
 $MID = INT [(BEG + END)/2]$

Step 2 : Repeat steps 3 and 4 while $BEG \leq END$
and $DATA [MID] \neq ITEM$

Step 3 : if $ITEM < DATA [MID]$ then
Set $END = MID - 1$
else

Set $BEG = MID + 1$

Step 4 : Set $MID = INT [(BEG + END)/2]$
[End of step2 Loop]

Step 5 : if $DATA [MID] = ITEM$ then
Set $LOC = MID$
else

Set $LOC = NULL$

Step 6 : Exit

এখানে,

$LB =$ Lower Bound, Data element গুলির প্রথম উপাদান।

$UB =$ Upper Bound, Data element গুলির শেষ উপাদান।

$MID =$ Data উপাদানগুলির মধ্য মান।

$ITEM =$ যে Element কে Search করতে হবে।

Binary Search পদ্ধতি : বাইনারি সার্চ পদ্ধতিতে কোনো একটি Array-কে সমান দুটি Segment-এ বিভক্ত করা হয়। অর্থাৎ, অ্যারের মধ্যবিন্দু নির্ণয় করা হয়। মধ্যবিন্দুর সাপেক্ষে বাম ও ডানদিকে দুটি Segment পাওয়া যায়। কোনো Array থেকে যে Item-টি সার্চ করে বের করতে হবে, অ্যারের মধ্য উপাদানের সাথে ঐ Item বা উপাদানটির তুলনা করা হয় এবং পর্যবেক্ষণ করা হয় যে, মধ্য উপাদানের সাপেক্ষে ঐ উপাদানটি ছোট, সমান বা বড়। যদি সমান হয়, তবে মধ্য স্থানটিই উপাদানের নির্ণীত স্থান। যদি ছোট হয়, তবে বাম Segment-এ কাক্ষিত উপাদানটি অবস্থান করতে পারে। আর যদি বড় হয়, তবে ডান Segment-এ কাক্ষিত উপাদানটি অবস্থান করতে পারে। যদি কাক্ষিত উপাদানটি বাম সেগমেন্টে অবস্থান করে, তবে বাম Segment-এ বিভক্ত করা হয় এবং পূর্বের পদ্ধতিই অবলম্বন করা হয়। এভাবে প্রতিবার Array কে দুটি ভাগে বিভক্ত করে সার্চ করা হয় বলে এ পদ্ধতিকে Binary search বলে।

অধ্যায়-৯

সটিং অপারেশন

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। **Sorting** বলতে কী বুঝ? বাকশির্ষো- ২০০৯, ১০, ১২, ১৫'পর, ১৭, ১৯

উত্তর : কোনো Data Structure হতে ডাটা গুলোকে মানের ক্রমানুসারে সাজানোকে Sorting বলে।

Note : Sorting দুই প্রকার। যথা :

- 1) Ascending order: যে Sorting পদ্ধতিতে data গুলিকে ছোট থেকে বড় মানের ক্রমানুসারে সাজানো হয়, তাকে Ascending order বলে। যেমন : A-Z, a-z, 0-9 ইত্যাদি।
- 2) Descending order : যে Sorting পদ্ধতিতে data গুলিকে বড় থেকে ছোট মানের ক্রমানুসারে সাজানো হয়, তাকে Descending order বলে। যেমন : Z-A, z-a, 9-0 ইত্যাদি।

*** ২। বিভিন্ন প্রকার **Sorting** পদ্ধতির নাম লেখ।

বাকশির্ষো- ২০০৭, ০৮, ০৮'পর, ০৯, ১৬'পর, ১৭'পর

উত্তর : বিভিন্ন প্রকার Sorting পদ্ধতির নাম নিম্নরূপ :

- | | |
|----------------------|--------------------|
| i) Insertion sort | ii) Selection sort |
| iii) Counting sort | iv) Bucket sort |
| v) Distribution sort | vi) Bubble sort |

vii) Quick sort

viii) Heap sort

ix) Merge sort

x) Radix sort ইত্যাদি।

***** ৩। Swapping কী?**

বাকশিবো- ২০০৬, ০৭, ০৮, ০৯, ১০, ১১, ১৩, ১৩'পরি, ১৬, ১৭, ১৭'পরি

উত্তর : Swapping মানে Interchange বা অদল বদল করা (স্থান বিনিময় করা)। যে পদ্ধতিতে দুটি ডাটার মধ্যে পারস্পরিক স্থান বিনিময় করা হয়, তাকে Swapping বলে।

যেমন : $a = 10$, $b = 20$, Swapping এর আগে

$a = 20$, $b = 10$, Swapping এর পরে।

ব্যবহার : i) Operating System এর Process প্রক্রিয়াকরণের ক্ষেত্রে (Main Memory ও Secondary Memory)

ii) Programming এ

iii) Sorting Algorithm

**** ৪। Heap Sort কাকে বলে?**

বাকশিবো- ২০০৭, ০৮, ০৮'পরি, ১১, ২১

উত্তর : Heap হচ্ছে এক বিশেষ ধরনের Tree, যেখানে Heap কে Sorting Algorithm হিসেবে ব্যবহার করা হয়, তাকে Heap Sort বলে। Heap sort দুই প্রকার। যথা :

i) Max Heap: Root এর মান সবচেয়ে বড়।

ii) Min Heap: Root এর মান সবচেয়ে ছোট।

**** ৫। Divide and Conquer Method বলতে কী বুঝায়?**

বাকশিবো- ২০০৯, ১৪'পরি, ২৩

উত্তর : Divide অর্থ বিভক্ত এবং Conquer অর্থ বিজয়। এই পদ্ধতিতে Unsorted data structure কে দুটি subset এ এমনভাবে ভাগ করা হয়, যাতে প্রথম set এর ডাটা সমষ্টি অপেক্ষা দ্বিতীয় set এর Data সমষ্টি sort থাকে। প্রতিটি subset দুই প্রান্ত হতে একইসাথে আলাদা সজ্জিত হতে থাকে।

Quick sort পদ্ধতি Divide and Conquer পদ্ধতিতে কাজ করে।

SOS সংক্ষিপ্ত প্রশ্নোত্তর :

***** ১। Bubble Sort পদ্ধতি বর্ণনা কর।**

BRTA- 2012, বাকশিবো- ২০০৪, ১৩'পরি, ১৫'পরি, ১৭,১৭'পরি

উত্তর : **Bubble Sort :** Bubble শব্দের অর্থ বুদবুদ। এ Sorting পদ্ধতিতে প্রথম দুটি সংখ্যার মধ্যে তুলনা করে। তুলনা করে দেখা হয়, কোন নাম্বার ছোট এবং কোনটি বড়। আমরা যদি ছোট থেকে বড় আকারে সাজাতে চাই, তাহলে যদি প্রথমটি থেকে দ্বিতীয়টি ছোট হয়, তাহলে প্রথম সংখ্যার জায়গায় দ্বিতীয়টি বসাই। দ্বিতীয় সংখ্যার জায়গায় প্রথম সংখ্যা বসাই। এরপর দ্বিতীয় সংখ্যার সাথে তৃতীয় সংখ্যা তুলনা করে। তৃতীয়টার সাথে চতুর্থ সংখ্যা, এভাবে শেষটা পর্যন্ত।

Bubble Sort Technique বা পদ্ধতি : একটি তালিকার উপাদানসমূহ $A[1], A[2], \dots, A[N]$ আকারে মেমরিতে সংরক্ষিত আছে। Bubble Sort অনুযায়ী নিচের পদ্ধতিতে উপাদানসমূহকে ছোট থেকে বড় আকারে সাজানো যায়-

Step-1: $A[1]$ এবং $A[2]$ এর মধ্যে Compare বা তুলনা করতে হবে। যদি $A[1] > A[2]$ হয়, তবে $A[1]$ এবং $A[2]$ Interchange বা Swapping হবে [Swapping মানে পরস্পরের স্থান বিনিময়]। যদি $A[1] < A[2]$ হয়, তাহলে কোনো পরিবর্তন হবে না [যেহেতু উর্ধ্বক্রম অনুসারে সাজানো হচ্ছে], এরপর $A[2]$ এবং $A[3]$ এর মধ্যে তুলনা করতে হবে।

Step-2: Step-1-কে পুনরাবৃত্তি করতে হবে, তবে এক্ষেত্রে শেষ উপাদান হবে $A[N - 2]$ । অর্থাৎ সবশেষে $A[N - 2]$ এবং $A[N - 1]$ এর মধ্যে তুলনা করে ডাটা $A[N - 2] < A[N - 1]$ অনুযায়ী সাজাতে হবে।

Step-2 অনুসরণ করার পর $A[N - 1]$ উপাদানটি দ্বিতীয় বৃহৎ ডাটা হবে। এক্ষেত্রে $[N - 2]$ সংখ্যক তুলনার প্রয়োজন হয়।

Step-3: Step-1-কে পুনরাবৃত্তি করতে হবে। এক্ষেত্রে শেষ উপাদান হবে $A[N - 2]$ । সবশেষে $A[N - 3]$ এবং $A[N - 2]$ এর মধ্যে তুলনা করে $A[N - 3] < A[N - 2]$ অনুযায়ী সাজাতে হবে।

Step-3 অনুসরণ করার পর $A[N - 2]$ উপাদানটি তৃতীয় বৃহৎ ডাটা হবে। এক্ষেত্রে $[N - 3]$ সংখ্যক তুলনার প্রয়োজন হয়।

Step-(N-1) : $A[1]$ এবং $A[2]$ -এর মধ্যে তুলনা করে ডাটা সাজাতে হবে যাতে $A[1] < A[2]$ হয়। (N-1) Step শেষে সমগ্র List-টি উর্ধ্বক্রম অনুসারে সজ্জিত হয়।

Bubble Sort পদ্ধতিতে প্রতিটি Step বা ধাপকে একটি Pass বলা হয়।

***** ২। Bubble Sort Algorithm লেখ।**

বাণিজ্য মন্ত্রণালয়- ২০২১, বাকাশিবো- ২০০২, ০৩, ০৪, ০৬, ০৭, ০৮, ০৮'পরি, ০৯, ১০, ১২

উত্তর : Algorithm :

Step 1: $K = 1$ to $N-1$ পর্যন্ত 2 এবং 3 step এর Repeat হবে।

Step 2: Set $PTR = 1$

Step 3: $PTR \leq N-K$ পর্যন্ত Loop এর Repeat হবে।

a) if $DATA [PTR] > DATA [PTR+1]$ হয়, তাহলে $DATA [PTR]$ এবং $DATA [PTR+1]$ এর মধ্যে পরস্পর স্থান বিনিময় (swapping) হবে। [End of if]

b) $PTR = PTR + 1$
[Inner Loop End]
[Step 1 outer Loop End]

Step 4: End

ব্যাখ্যা :

1) Bubble Sort এর ক্ষেত্রে, N সংখ্যক Data এর জন্য $N-1$ সংখ্যক তুলনা করতে হয় বলে K এর মান 1 থেকে $n-1$ পর্যন্ত।

2) প্রথম Data হতে তুলনা শুরু হবে, তাই $PTR = 1$

3) প্রতি Pass এ Iteration সংখ্যা মোট সংখ্যার চেয়ে 1 কম। তাই PTR এর মান $\leq N-K$

a) যদি পাশাপাশি দুটি মানের মধ্যে ডানপাশে ছোট মান থাকে (Ascending order এর ক্ষেত্রে), তবে মানের swapping হবে। আর যদি ডানপাশে বড় মান থাকে, তবে swapping হবে না।

b) পরবর্তী মানের সাথে আবার তুলনা করতে হবে। এভাবে সর্বশেষ মান পর্যন্ত চলতে থাকবে।

4) Algorithm এর সমাপ্তি।

*** ৩। Bubble Sort এর Complexity নির্ণয় কর।

বাকশির্বো- ২০০৪, ০৬, ০৭, ০৮, ১০, ১১, ১৬

উত্তর : কোনো Algorithm এর Complexity দুই ধরনের হয়ে থাকে।

i) Time Complexity: Algorithm Run হতে প্রয়োজনীয় সময়

ii) Space Complexity: Algorithm মেমরিতে কতটুকু জায়গা বরাদ্দ করে।

Bubble sort প্রথম পাসে $n - 1$ সংখ্যক তুলনার দ্বারা সর্ববৃহৎ সংখ্যাটি নির্ণয় করা হয়। দ্বিতীয় পাসে $n - 2$ সংখ্যক তুলনার দ্বারা দ্বিতীয় বৃহৎ সংখ্যাটি নির্ণয় করা হয়। একই প্রক্রিয়া পর্যায়ক্রমে অবলম্বন করে Bubble sort সম্পন্ন করা হয়।

Bubble sort এ প্রয়োজনীয় তুলনার সংখ্যা,

$$f(n) = (n-1) + (n-2) + (n-3) + \dots + 3 + 2 + 1$$

$$= \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2}$$

$$= \frac{n^2}{2} + 0(n)$$

$$\therefore f(n) = O(n^2)$$

১ম Pass এ Iteration $n-1$

২য় এ Pass এ Iteration $n-2$

৩য় এ Pass এ Iteration $n-3$

* 8 | Quick Sort এর Complexity নির্ণয় কর।

বাকাশিবো- ২০০৪, ০৬, ১১, ২৩

উত্তর : কোনো Algorithm এর Complexity দুই ধরনের হয়ে থাকে।

i) Time Complexity: Algorithm Run হতে প্রয়োজনীয় সময়।

ii) Space Complexity: Algorithm মেমরিতে কতটুকু জায়গা বরাদ্দ করে।

Worst Case : যদি কোনো তালিকা পূর্ব থেকে সাজানো থাকে, তাহলে Worst Case সৃষ্টি হয়। এ পর্যায় প্রথম উপাদানের প্রথম অবস্থান শনাক্ত করার জন্য অ্যালগরিদমের n সংখ্যক তুলনার প্রয়োজন হয়। এক্ষেত্রে প্রথম Sublist-এ কোনো উপাদান থাকবে না, কিন্তু দ্বিতীয় Sublist-এ $n-1$ সংখ্যক উপাদান থাকবে।

একইভাবে, দ্বিতীয় উপাদানের অবস্থান শনাক্ত করার জন্য $n-1$ সংখ্যক তুলনার প্রয়োজন পড়বে। তৃতীয় উপাদানের জন্য প্রয়োজন হবে $n-2$ সংখ্যক তুলনা এবং এভাবে ধারাবাহিকভাবে সিটিং-এর তুলনা চলতে থাকবে।

$$\begin{aligned}\therefore f(n) &= n + (n - 1) + \text{-----} + 2 + 1 \\ &= \frac{n(n+1)}{2} = \frac{n^2}{2} + 0(n) \\ \therefore f(n) &= O(n^2)\end{aligned}$$

Average Case: Average Case-এ Quick Sort অ্যালগরিদমের Complexity $f(n) = O(n \log_n)$ এবং এটি পাওয়া যায় যখন প্রত্যেক

Reduction Step-এ Algorithm দুটি সাব লিস্ট তৈরি করে। ফলে Average case-এ নিচের নিয়মগুলো অনুসরণ করা হয়-

- i) একটি উপাদান স্থাপন করার জন্য মূল List দুটি Sublist-এ পরিণত হয়।
- ii) দুটি উপাদানকে স্থির করার জন্য দুটি সাব লিস্টকে চারটি Sublist-এ বিভক্ত করা হয়।
- iii) চারটি উপাদানকে স্থির করার জন্য চারটি Sublist-কে আটটি Sublist-এ বিভক্ত করা হয়।
- iv) আটটি উপাদানকে স্থির করার জন্য আটটি Sublist কে ষোলটি Sublist-এ বিভক্ত করা হয় এবং এভাবে পর্যায়ক্রমে কার্য সম্পাদন করা হয়। এখন লক্ষ্য করা যাচ্ছে, K সংখ্যক ধাপ অবলম্বন করে 2^{k-1} টি উপাদান সাজানো যায়। সুতরাং, প্রায় $\log_2^n$ সংখ্যক ধাপ অবলম্বন করতে হয়।

সেজন্য Average ক্ষেত্রে প্রয়োজনীয় সংখ্যক তুলনা বা প্রয়োজনীয় নির্বাহ সময়কে নিম্নরূপে প্রকাশ করা যায়-

$$f(n) = O(n \log n)$$

যেখানে, O হচ্ছে Big O notation

$f(n)$ - এর ব্যবহারিক মান হচ্ছে $1.4 [n \log n]$

**** ৫। Merge Sort Algorithm লেখ।**

Bcs- 36th, বিভিন্ন মন্ত্রণালয়- ২০১৯, বাকাশিবো- ২০০৫, ০৭, ০৯, ০৯'পরি, ১৫'পরি

উত্তর : Algorithm :

Step 1: $L=1$ set করা হবে।

Step 2: $L < N$ পর্যন্ত Step 3 & 4 Repeat হবে।

Step 3: pass (A, N, L, B) কে call

Step 4: pass (B, N, $2 * L$) কে call

Step 5: $L = 4 * L$ set করা হবে।

Step 6: End

ব্যাখ্যা :

1) $L = \text{length}$, ১ম অবস্থায় 1 সেট করে।

2) $N = \text{মোট Data}$, যতক্ষণ পর্যন্ত L এর মান N এর চেয়ে ছোট থাকবে, ততক্ষণ পর্যন্ত 3 ও 4 নং step Repeat হবে।

3) pass (A, N, L, B), $A = \text{Array}$

$N = \text{মোট উপাদান}$

$L = \text{প্রতিবার কতগুলি করে ডাটা নিচ্ছি}$

$B = \text{নতুন sorted Array}$

4) Pass (B, N, $2 * L$), আগের চেয়ে দ্বিগুণ আকারে ডাটা একত্রিত করা হচ্ছে।

5) L এর মান চারগুণ করে নিয়ে একত্রিত করা হচ্ছে।

রচনামূলক প্রশ্নোত্তর :

বাকাশিবো- ২০০৪, ০৫, ০৭, ০৮, ১৩, ১৩'পরি, ২২

Scan করতে হবে ডান হতে বামে। Pivot এর চেয়ে ছোট মান পেলেই Pivot ও ঐ ছোট মানের সাথে Swapping হবে।

Right

মধ্যে স্থান বিনিময় করবে। ফলে নতুন list-টি দাঁড়ায়-

Right
pivot

এখন যেহেতু Pivot point, Right side-এ তাই scan শুরু হবে Left থেকে এবং যতক্ষণ না Left এর মান Pivot এর চেয়ে বড় হবে, ততক্ষণ Scan করবে।

$40 > 44$? No, তাহলে Left এর মান 1 বেড়ে যাবে। এখন Left = 31 হবে এবং পুনরায় তুলনা করা হয়। এভাবে Left এর মান যখন 55 তখন $55 > 44$? Yes, তাহলে 55 এবং 44 এর মধ্যে স্থান বিনিময় হবে। ফলে নতুন Left- টি দাঁড়ায়-

40, 31, 11, ④④, 77, 90, ⑤⑤, 60, 99

↑ ↑

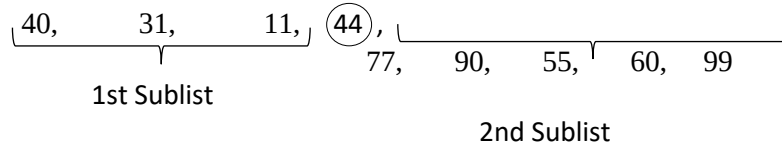
Left Right

pivot

এবার 55 থেকে শুরু করে বামদিকে Scan করবে এবং তুলনা করা হবে Right এর মান Pivot এর চেয়ে ছোট কি না। এভাবে 55, 90, 77 এর সাথে তুলনা হবে কিন্তু Right এর মান Pivot-এর চেয়ে ছোট পাওয়া যায় না কারণ $77 > 44$. এর অর্থ হচ্ছে উপরোক্ত পদ্ধতিতে সমস্ত সংখ্যাটি Scan হবার পর 44 এর সাথে তুলনা সম্পন্ন হবে এবং Left = Right = Pivot হবে।

অর্থাৎ 44 সংখ্যাটি তার নিজের সঠিক অবস্থানে আছে।

এখন দেখা যাচ্ছে, 44 এর বাম পার্শ্বস্থ Sublist এর প্রতিটি সংখ্যা 44 অপেক্ষা ছোট। পক্ষান্তরে, ডান পার্শ্বস্থ Sublist-এর প্রতিটি সংখ্যা 44 অপেক্ষা বড়। নিচে Sublist-দ্বয় দেখানো হয়েছে।



সুতরাং এ ব্যবস্থায় 44 সঠিক অবস্থানে স্থাপিত হয়। ফলে মূল A List-কে দুটি Sublist-এ বিভক্ত (Divide) করে Sorting-এর মাধ্যমে সামগ্রিকভাবে Sorting প্রক্রিয়াকে কমিয়ে আনা সম্ভব হয়।

উপরোক্ত Reduction step দুই বা ততোধিক উপাদান সম্পন্ন প্রতিটি Sublist কে Repeat করা হয়। যেহেতু যেকোনো সময় কেবলমাত্র একটি Sublist-কে প্রক্রিয়া করা যায়, কাজেই এ পদ্ধতিতে Sublist-কে সাময়িকভাবে সংরক্ষণের জন্য দুটি Stack (LOWER এবং UPPER) ব্যবহৃত হয়।

প্রতিটি Sublist-এর প্রথম ও শেষ উপাদানের Address কে Boundary value বলে। এগুলোকে প্রথমে LOWER এবং UPPER Stack-এ PUSH করা হয়। অতপর Stack থেকে Top boundary value-বলে অপসারণ করে সংশ্লিষ্ট Sublist-এ Reduction step প্রয়োগ হয় Sorting করার জন্য।

*** ৩। **Bubble Sort Algorithm** লিখে বর্ণনা কর।

বাকাশিৰো- ২০০৪, (০৮-১২), ০৯'পরি, ১০'পরি, ১৪, (১৪-১৮)'পরি

উত্তর : **Algorithm :**

Step 1: $K = 1$ to $N-1$ পর্যন্ত 2 এবং 3 step এর Repeat হবে।

Step 2: set $PTR = 1$

Step 3: $PTR \leq N-K$ পর্যন্ত Loop এর Repeat হবে।

c) if $DATA [PTR] > DATA [PTR+1]$ হয়, তাহলে
 $DATA [PTR]$ এবং $DATA [PTR+1]$ এর মধ্যে
পরস্পর স্থান বিনিময় (swapping) হবে। [End of if]

d) $PTR = PTR+1$
[Inner Loop End]
[step 1 outer Loop End]

Step 4: End

ব্যাখ্যা :

1) Bubble sort এর ক্ষেত্রে, N সংখ্যক Data এর জন্য $N-1$ সংখ্যক তুলনা করতে হয় বলে K এর মান 1 থেকে $N-1$ পর্যন্ত।

2) প্রথম Data হতে তুলনা শুরু হবে, তাই $PTR = 1$

3) প্রতি Pass এ Iteration সংখ্যা মোট সংখ্যার চেয়ে 1 কম। তাই PTR এর মান $\leq n-k$

a) যদি পাশাপাশি দুটি মানের মধ্যে ডানপাশে ছোট মান থাকে (Ascending order এর ক্ষেত্রে), তবে মানের Swapping হবে। আর যদি ডানপাশে বড় মান থাকে, তবে Swapping হবে না।

b) পরবর্তী মানের সাথে আবার তুলনা করতে হবে। এভাবে সর্বশেষ মান পর্যন্ত চলতে থাকবে।

4) Algorithm এর সমাপ্তি।

Bubble Sort : Bubble শব্দের অর্থ বুদবুদ। এ Sorting পদ্ধতিতে প্রথম দুটি সংখ্যার মধ্যে তুলনা করে। তুলনা করে দেখা হয়, কোন নাম্বার ছোট এবং কোনটি বড়। আমরা যদি ছোট থেকে বড় আকারে সাজাতে চাই, তাহলে যদি প্রথমটি থেকে দ্বিতীয়টি ছোট হয়, তাহলে প্রথম সংখ্যার জায়গায় দ্বিতীয়টি বসাই। দ্বিতীয় সংখ্যার জায়গায় প্রথম সংখ্যা বসাই। এরপর দ্বিতীয় সংখ্যার সাথে তৃতীয় সংখ্যা তুলনা করে। তৃতীয়টার সাথে চতুর্থ সংখ্যা, এভাবে শেষটা পর্যন্ত।

Bubble Sort Technique বা পদ্ধতি : একটি তালিকার উপাদানসমূহ $A[1], A[2], \dots, A[N]$ আকারে মেমরিতে সংরক্ষিত আছে। Bubble Sort অনুযায়ী নিচের পদ্ধতিতে উপাদানসমূহকে ছোট থেকে বড় আকারে সাজানো যায়-

Step-1: $A[1]$ এবং $A[2]$ এর মধ্যে Compare বা তুলনা করতে হবে। যদি $A[1] > A[2]$ হয়, তবে $A[1]$ এবং $A[2]$ Interchange বা Swapping হবে [Swapping মানে পরস্পরের স্থান বিনিময়]। যদি $A[1] < A[2]$ হয়, তাহলে কোন পরিবর্তন হবে

না [যেহেতু উর্ধ্বক্রম অনুসারে সাজানো হচ্ছে], এরপর $A[2]$ এবং $A[3]$ এর মধ্যে তুলনা করতে হবে।

Step-2: Step-1-কে পুনরাবৃত্তি করতে হবে, তবে এক্ষেত্রে শেষ উপাদান হবে $A[N - 2]$ । অর্থাৎ সবশেষে $A[N - 2]$ এবং $A[N - 1]$ এর মধ্যে তুলনা করে ডাটা $A[N - 2] < A[N - 1]$ অনুযায়ী সাজাতে হবে।

Step-2 অনুসরণ করার পর $A[N - 1]$ উপাদানটি দ্বিতীয় বৃহৎ ডাটা হবে। এক্ষেত্রে $[N - 2]$ সংখ্যক তুলনার প্রয়োজন হয়।

Step-3: Step-1-কে পুনরাবৃত্তি করতে হবে। এক্ষেত্রে শেষ উপাদান হবে $A[N - 2]$ । সবশেষে $A[N - 3]$ এবং $A[N - 2]$ এর মধ্যে তুলনা করে $A[N - 3] < A[N - 2]$ অনুযায়ী সাজাতে হবে।

Step-3 অনুসরণ করার পর $A[N - 2]$ উপাদানটি তৃতীয় বৃহৎ ডাটা হবে। এক্ষেত্রে $[N - 3]$ সংখ্যক তুলনার প্রয়োজন হয়।

Step-(N-1) : $A[1]$ এবং $A[2]$ -এর মধ্যে তুলনা করে ডাটা সাজাতে হবে যাতে $A[1] < A[2]$ হয়। (N-1) Step শেষে সমগ্র List-টি উর্ধ্বক্রম অনুসারে সজ্জিত হয়।

Bubble Sort পদ্ধতিতে প্রতিটি Step বা ধাপকে একটি Pass বলা হয়।

বাংলাদেশ কারিগরি শিক্ষা বোর্ড, ঢাকা
ডিপ্লোমা ইন ইঞ্জিনিয়ারিং
৪র্থ পর্ব সমাপনী পরীক্ষা- ২০২৩
টেকনোলজি: কম্পিউটার সায়েন্স (২০২২ প্রবিধান)
বিষয়: ডাটা স্ট্রাকচার অ্যান্ড অ্যালগরিদম
বিষয় কোড: ২৮৫৪২

সময়: ৩ ঘন্টা

পূর্ণমান: ৬০

ক ও খ-বিভাগের সকল প্রশ্নের এবং গ-বিভাগের যে কোন ৫ (পাঁচ)
টি প্রশ্নের উত্তর দাও ।

ক-বিভাগ (মান: $১ \times ১০ = ১০$)

- ১। ডাটা Type গুলো উল্লেখ কর ।
- ২। সকল “Leaf-ই-Node, কিন্তু সকল Node-ই-Leaf নয়” কেন?
- ৩। Linked List কী?
- ৪। $(x + y)xz$ -এর Post fix Notation লেখ ।
- ৫। Enumeration কী?
- ৬। Location নির্ণয়ের সূত্রটি লেখ, যা Linear Array-এর K-তম উপাদান ।
- ৭। DE-Queue কী?
- ৮। Complexity নির্ণয়ের ক্ষেত্র কয়টি ও কী কী?
- ৯। Spanning Tree কী?
- ১০। Subroutine কাকে বলে?

খ-বিভাগ (মান: $২ \times ১০ = ২০$)

- ১১। Polish Notation বলতে কী বোঝায়?
- ১২। Quick Sort Algorithm-এর Complexity লেখ।
- ১৩। Traversing ও Searching-এর মাঝে পার্থক্য লেখ।
- ১৪। $(4x-9b)^2 (A-7B) + 2C$ কে Tree আকারে প্রকাশ কর।
- ১৫। Divide ও Conquer বলতে কী বোঝায়?
- ১৬। Swapping কী? Swapping-এর দুটি ব্যবহার লেখ।
- ১৭। Algorithm-এর বৈশিষ্ট্য লেখ।
- ১৮। LIFO ও FIFO-এর মাঝে পার্থক্য লেখ।
- ১৯। সংজ্ঞা লেখ : (i) Root; (ii) Leaf; (iii) Keys; (iv) Sub-tree.
- ২০। কোন Stack-এ Item Push করার Procedure লেখ।

গ-বিভাগ (মান: $৬ \times ৫ = ৩০$)

- ২১। বিভিন্ন প্রকার Tree-এর Data Structure চিত্রসহ বর্ণনা কর।
- ২২। Matrix Multiplication-এর Algorithm লেখ।
- ২৩। Linked list-এ Data সংযোজন Algorithm লেখ।
- ২৪। উদাহরণসহ Infix, Prefix, Postfix Notation বর্ণনা কর।
- ২৫। দ্বি-মাত্রিক Array কে মেমরিতে উপস্থাপন পদ্ধতি বর্ণনা কর।
- ২৬। Diagram সহ বিভিন্ন প্রকারের Dimensional-Array বর্ণনা কর।